



SentinelFW

Local nftables Firewall Manager & Security Monitor

Final Project Report

Lucky Joshi

10 October 2026

Version 1.0.0

160 tests · all passing

72% coverage

Python 3.10+

Kali Linux

Repository: github.com/Lucky-Joshi/SentinelFW · MIT License

Abstract

SentinelFW is a local nftables firewall manager and security monitor for Linux, developed and tested on Kali Linux. It was built to address a specific gap: managing a packet filter and understanding what that filter is actually doing are two separate skills, and the tools available to a learner usually force a choice between one and the other. SentinelFW joins them.

The tool keeps its firewall rules in a human-readable `rules.yaml` file rather than applying them directly, and it never lets a rule change reach the kernel until the operator explicitly runs `firewall apply`. Before anything is installed, SentinelFW prints the *complete* `nft` script it intends to run, takes a backup of the live ruleset, asks for confirmation, applies the script as a single atomic transaction, and then re-reads the kernel ruleset to verify that what it installed is what it intended. Every rule it creates is confined to one object it owns — `table inet sentinelfw`, chain `input`, a base chain at priority `-10` with policy `accept` — so it can add filtering without ever becoming the reason a machine stops answering.

On the monitoring side, SentinelFW reads the log lines that its own rules produce, stores them in a SQLite database, and runs five threshold-and-window detectors over them: port scans, brute-force attempts, repeated blocks on a closed port, connection spikes, and probes against sensitive services. Findings are de-duplicated by fingerprint, carry the evidence that produced them, and are explained in plain language with concrete next steps. A terminal dashboard, a report generator in three formats, and a small knowledge base for ports and attack types round out the tool.

The project comprises **9,248 lines of application code** across 23 source modules and **1,313 lines** of tests. The test suite contains **160 tests**, all of which pass in approximately **51 seconds** with an overall statement coverage of **72%**. The suite runs entirely against temporary directories and never invokes `nft`; the root-only paths are exercised by asserting that they refuse rather than by applying rules.

This report documents the motivation, design, architecture, implementation, testing methodology, measured results, security analysis, limitations and future direction of SentinelFW. It is intended to serve as the final technical report for the project and as a reference for anyone evaluating or extending it.

Acknowledgement

This project builds directly on the work of the open-source community. It would not exist without **nftables** and its `nft` command-line interface, the **Linux kernel** netfilter subsystem that produces the log lines SentinelFW reads, **SQLite** for embedded storage, the **Python** language and standard library, and the **Rich** library for terminal rendering. The design also learned from the safety posture of mature firewall front-ends: preview first, back up always, confirm explicitly.

Declaration

I declare that SentinelFW is my own original work. It was designed, implemented, tested and documented by me. All external libraries and tools used are acknowledged above and in the References section, and no part of this work misrepresents the contribution of others.

Table of Contents

Abstract	2
Acknowledgement	3
Declaration	4
Table of Contents	5
List of Figures	7
List of Tables	8
1. Introduction	9
1.1 What SentinelFW is	9
1.2 Why it exists	9
1.3 Scope	9
1.4 Report organisation	9
2. Problem Statement	10
3. Objectives	11
4. Literature Review and Comparison	12
5. System Requirements	13
6. Technology Stack	14
7. System Architecture	15
7.1 Layering rules	15
7.2 Module interaction	15
7.3 Data flow	16
8. UML and Design Diagrams	17
8.1 Use case view	17
8.2 firewall apply control flow	18
8.3 Detection sequence	20
8.4 Monitoring activity	20
8.5 Deployment view	22
9. Database Design	23
9.1 Entity-relationship diagram	23
9.2 Tables	23
9.3 Retention	24
10. Project Folder Structure	25
11. Module Description	26
11.1 The detector set	26
12. Implementation Details	28
12.1 The safety invariants	28
12.2 Applying rules	28
12.3 Rule ordering and conflicts	28
12.4 Parsing without crashing	29
13. Configuration	30
14. Interface Design	31
14.1 Environment check	31
14.2 Recording and previewing a rule	31

14.3 Monitoring and detection	33
15. Command Reference	39
16. Exit Codes	40
17. Test Plan	41
17.1 Strategy	41
17.2 Levels of testing	41
17.3 Environment	41
17.4 Test data isolation	42
18. Test Cases and Results	43
18.1 Rule validation and injection	43
18.2 Configuration	44
18.3 Storage and detection	45
18.4 End-to-end CLI	46
18.5 Safety and containment	47
18.6 Summary	48
19. Performance Analysis	49
19.1 In-process latency	49
19.2 End-to-end CLI latency	49
19.3 Scaling characteristics	50
20. Security Analysis	51
20.1 Threat model	51
20.2 Controls	51
20.3 What SentinelFW does not protect	52
21. Code Coverage	53
22. Limitations	55
23. Future Enhancements	56
24. Conclusion	57
25. References	58
Appendix A — Installation	59
Appendix B — First Session	60
Appendix C — Sample Configuration	61
Appendix D — Full Test Inventory	62
Appendix E — Sample Logs and Evidence	67

List of Figures

Figure 1 — Layered system architecture	15
Figure 2 — Component and dependency diagram	16
Figure 3 — Use case diagram	17
Figure 4 — firewall apply control flow	19
Figure 5 — Detection sequence diagram	20
Figure 6 — Monitoring activity diagram	21
Figure 7 — Deployment diagram	22
Figure 8 — Database entity-relationship diagram	23
Figure 9 — Code coverage by module	53
Figure 10 — Test distribution across the suite	54
Figure 11 — In-process operation latency	49
Figure 12 — <code>sentinel fw doctor</code> environment check	31
Figure 13 — Recording a block rule	32
Figure 14 — Listing stored rules	32
Figure 15 — Previewing the exact nft script	33
Figure 16 — Synthetic traffic demonstration	34
Figure 17 — Live terminal dashboard	35
Figure 18 — Alerts with plain-language explanations	36
Figure 19 — Generated security report	37
Figure 20 — explain port 22	38

List of Tables

Table 1 — System and software requirements.	13
Table 2 — Technology stack.	14
Table 3 — Module responsibilities	26
Table 4 — Principal configuration defaults.	30
Table 5 — Detectors and default thresholds	26
Table 6 — Exit codes	40
Table 7 — Command reference	39
Table 8 — Safety invariants and enforcement.	28
Table 9 — Test environment.	41
Table 10 — Rule validation and injection test results.	43
Table 11 — Configuration test results	44
Table 12 — Storage and detection test results	45
Table 13 — End-to-end CLI test results	46
Table 14 — Safety and containment test results	47
Table 15 — Test summary by module	48
Table 16 — In-process latency measurements.	49
Table 17 — End-to-end CLI command latency	49
Table 18 — Coverage by module	53
Table 19 — Security controls and their tests	51
Table 20 — Known limitations	55
Table 21 — Future enhancements	56

1. Introduction

1.1 What SentinelFW is

SentinelFW is a command-line tool that manages a single nftables chain and monitors the traffic that chain sees. It is written in Python, requires no compiled components, and runs straight from a cloned directory or as an installed `sentinelfw` command. It presents nine groups of subcommands — `firewall`, `monitor`, `alerts`, `report`, `dashboard`, `explain`, `db`, `config`, and `doctor` — behind one entry point.

The tool is deliberately narrow. It does not try to be a complete firewall. It manages *one* table, `table inet sentinelfw`, adds filtering at an early priority so that its drops take effect before most services see the packet, and uses an `accept` policy so that traffic no rule matches carries on unaffected. It cannot lock an operator out of their own machine, which is the failure mode that makes firewall tooling frightening.

1.2 Why it exists

A firewall change is one of the few routine actions that can render a machine unreachable. Yet most beginner-facing tooling hides the command it runs, applies rules straight from a text file with no snapshot, and offers no route back if the result was wrong. At the same time, the reason a learner wants a firewall at all is usually to see what is happening: which addresses are knocking, which ports are being probed, whether the block they added is actually doing anything.

SentinelFW was written to close both gaps at once. It makes firewall changes inspectable and reversible, and it turns the firewall's own logs into explanations a learner can act on.

1.3 Scope

In scope: rule management for addresses, networks and TCP/UDP ports; safe, atomic application of those rules; ingestion of the firewall's log lines from the systemd journal, from log files, or from a synthetic demo stream; storage of events in SQLite; five detectors; alerting with de-duplication; a terminal dashboard; text, Markdown and JSON reports; and a knowledge base for ports and attack patterns.

Out of scope: full-ruleset authoring, packet capture or deep packet inspection, signature-based intrusion detection, remote or multi-host management, and any form of machine learning.

1.4 Report organisation

Section 2 states the problem. Section 3 lists objectives. Section 4 compares SentinelFW briefly with existing tools. Sections 5–7 cover requirements, technology and architecture. Sections 8–10 present the UML views, database design and folder structure. Sections 11–13 describe the modules and the key implementation workflows. Section 14 shows the interface. Sections 15–17 document the test plan, the results of all 160 tests, and measured performance. Sections 18–20 analyse security and state limitations and future work. Section 21 concludes.

2. Problem Statement

Managing nftables correctly is hard for four reasons.

First, the command surface is unforgiving. nftables rules are applied with `nft`, and the same command that installs a rule can, with a small mistake, flush an entire ruleset. The difference between `flush chain` and `flush table` is the difference between clearing your own rules and destroying everything — including the rules Docker and UFW rely on. Tools that hide the command being run remove the operator's ability to notice that difference.

Second, changes are rarely reversible by default. Most approaches write a ruleset file and load it, with no snapshot of what was there before. If the result is wrong, there is nothing to roll back to.

Third, the firewall is noisy but opaque. A drop produces a kernel log line. Those lines accumulate in the journal, and reading a wall of `SRC=... DST=... DPT=...` tells an experienced administrator a great deal and tells a learner almost nothing. There is no shortage of logs; there is a shortage of interpretation.

Fourth, existing security tools assume expertise. Full IDS platforms are powerful but heavy and configured by specialists. The beginner is left choosing between a single `nft` command they do not fully understand and a platform they cannot reasonably run.

SentinelFW addresses these points directly: it separates the *decision* to change a rule from the *act* of changing it, it backs up before it acts and verifies after, it summarises and explains the traffic the firewall logs, and it keeps the whole surface small enough to read in an afternoon.

3. Objectives

1. **Manage firewall rules safely.** Provide a small, validated vocabulary for blocking and allowing addresses, networks and ports, stored in a human-readable file.
2. **Never surprise the kernel.** Make rule changes and kernel changes separate, explicit steps, preview the exact command, back up, confirm, apply atomically, and verify.
3. **Contain blast radius.** Modify exactly one nftables object and refuse anything else.
4. **Monitor firewall activity.** Ingest the firewall's own log lines from realistic sources and store them durably.
5. **Detect suspicious behaviour.** Apply transparent, tunable heuristics that a learner can reason about.
6. **Explain, not just count.** Turn events and findings into plain-language descriptions with recommended actions.
7. **Report.** Produce human and machine-readable summaries of security state.
8. **Be a good citizen.** Run without root where possible, drop privileges where not needed, and require explicit consent for every mutation.
9. **Be testable without a live firewall.** Verify behaviour against temporary directories and never touch the kernel during tests.

4. Literature Review and Comparison

SentinelFW sits in a crowded space, but at an unusual point in it. The table below compares it with the tools a learner is most likely to encounter. The comparison is qualitative and reflects documented scope, not benchmarked performance.

Tool	Category	Strengths	How SentinelFW differs
<code>nft</code> / <code>nftables</code>	Packet filter	The real thing; complete control; kernel-native	SentinelFW is a <i>manager in front of</i> one chain, adding preview, backup, explanation and monitoring; not a replacement
UFW	Front-end	Simple allow/deny; widely documented	UFW owns the whole ruleset; SentinelFW owns one table, coexists with UFW/Docker, and adds detection
firewalld	Front-end	Zones, services, dynamic rules	Service-level abstraction; SentinelFW is address/port focused and log-centric rather than zone-centric
Fail2ban	Reactive blocker	Parses logs and auto-bans	Fail2ban reacts; SentinelFW <i>detects and explains</i> but does not silently ban, keeping a human in the loop
Suricata / Snort	IDS/IPS	Signature and protocol analysis on live traffic	Packet-based and heavy; SentinelFW is log-based, lightweight, and heuristic
ELK / Wazuh	SIEM	Aggregation, dashboards, correlation at scale	Multi-host and infrastructure-level; SentinelFW is single-host, local-first, and dependency-light

The gap SentinelFW fills is the *learning* gap: a tool whose generated command is always shown, whose detection rules are readable, whose findings come with explanations, and which is safe enough to run on the same machine you are learning on.

5. System Requirements

Table 1 — System and software requirements.

Category	Requirement	Notes
Operating system	Linux	Developed and tested on Kali Linux (Debian-based)
Python	3.10 or newer	Tested on CPython 3.14.7
Python packages	<code>rich</code> $\geq$ 13, <code>PyYAML</code> $\geq$ 6	Also available as distro packages
System package	<code>nftables</code>	Only needed to touch the live ruleset
Storage	~5 MB on disk	SQLite database grows with events
Privileges	Root only for <code>apply</code> , <code>validate</code> , <code>backup</code> , <code>restore</code> , and journal reads	Everything else runs unprivileged
Optional	Membership in <code>systemd-journal</code> group	To read the kernel journal without root
Hardware	Any modern x86-64/ARM Linux host	No GPU, no packet-capture NIC

The test environment used for this report is summarised in Table 9.

6. Technology Stack

Table 2 — Technology stack.

Layer	Technology	Role in SentinelFW
Language	Python 3.10+	Entire application
Packet filter	nftables / <code>nft</code>	The kernel ruleset SentinelFW manages and logs
Persistence	SQLite 3.53	Events, alerts, rule-hit counters, metadata
Configuration	YAML via <code>PyYAML.safe_load</code>	<code>config.yaml</code> and <code>rules.yaml</code>
Terminal UI	Rich 15	Tables, panels, colour, live dashboard
Logging	Python <code>logging</code> + <code>RotatingFileHandler</code>	<code>logs/sentinelfw.log</code> , including every <code>nft</code> call
Log sources	<code>journalctl</code> , plain files, stdin, synthetic demo	Input to the parser
Process execution	<code>subprocess.run</code> with argument lists	The only way an external command is run
Packaging	<code>setuptools</code> / <code>pyproject.toml</code>	<code>sentinelfw</code> console entry point
Testing	pytest 9	160 tests, plus <code>pytest-cov</code> for coverage

Design choices of note:

- **`yaml.safe_load` only.** Configuration and rules are data, never code, so a hostile YAML file cannot execute.
- **No shell, ever.** Every external command is executed as an argument list; no string is handed to a shell to interpret.
- **Standard library first.** `ipaddress`, `sqlite3`, `subprocess`, `re`, `threading`, `argparse` and `dataclasses` carry most of the load; the only third-party runtime dependencies are Rich and PyYAML.

7. System Architecture

SentinelFW is organised into four layers. The presentation layer is what the operator types and sees; the service layer orchestrates commands; the domain layer validates, renders and parses; and the infrastructure layer is the set of external systems and persisted state the tool depends on.

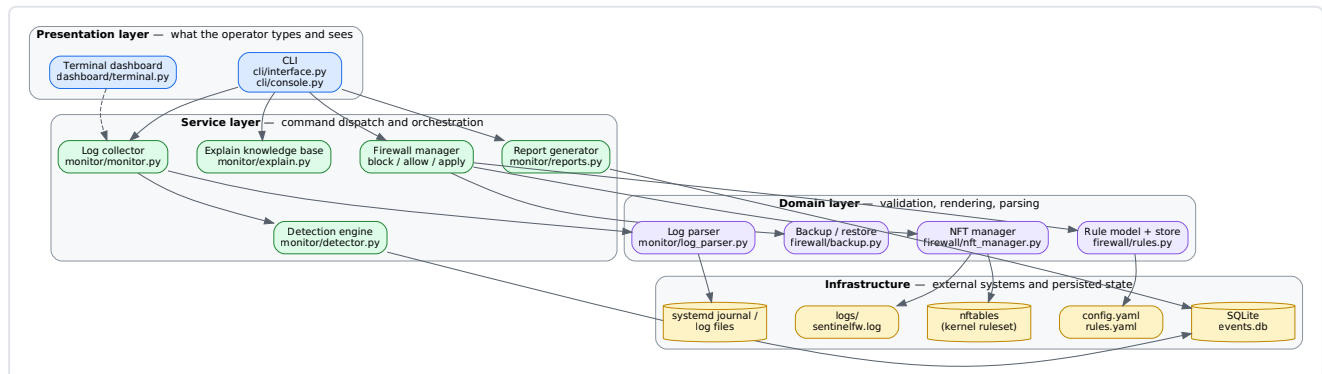


Figure 1 — Layered system architecture of SentinelFW.

7.1 Layering rules

- The **presentation layer** never talks to the kernel or the database directly except through the service layer (the dashboard reads the database for display, which is a deliberate, read-only exception).
- The **service layer** owns policy decisions: whether to apply, whether to detect, whether to persist.
- The **domain layer** owns correctness: validation, nft rendering, log parsing, rule ordering and conflict detection.
- The **infrastructure layer** is the only place where `nft`, files, SQLite and the journal are actually touched.

7.2 Module interaction

Figure 2 shows the concrete module dependencies. `cli.interface` is the hub: it parses arguments, loads configuration once, and dispatches to the domain and service modules. The firewall modules depend on configuration for identifiers but never on the CLI. The monitoring modules depend on the database and the explain knowledge base. Cross-cutting concerns — errors, logging, formatting — live in `exceptions.py`, `logsetup.py` and `utils.py`.

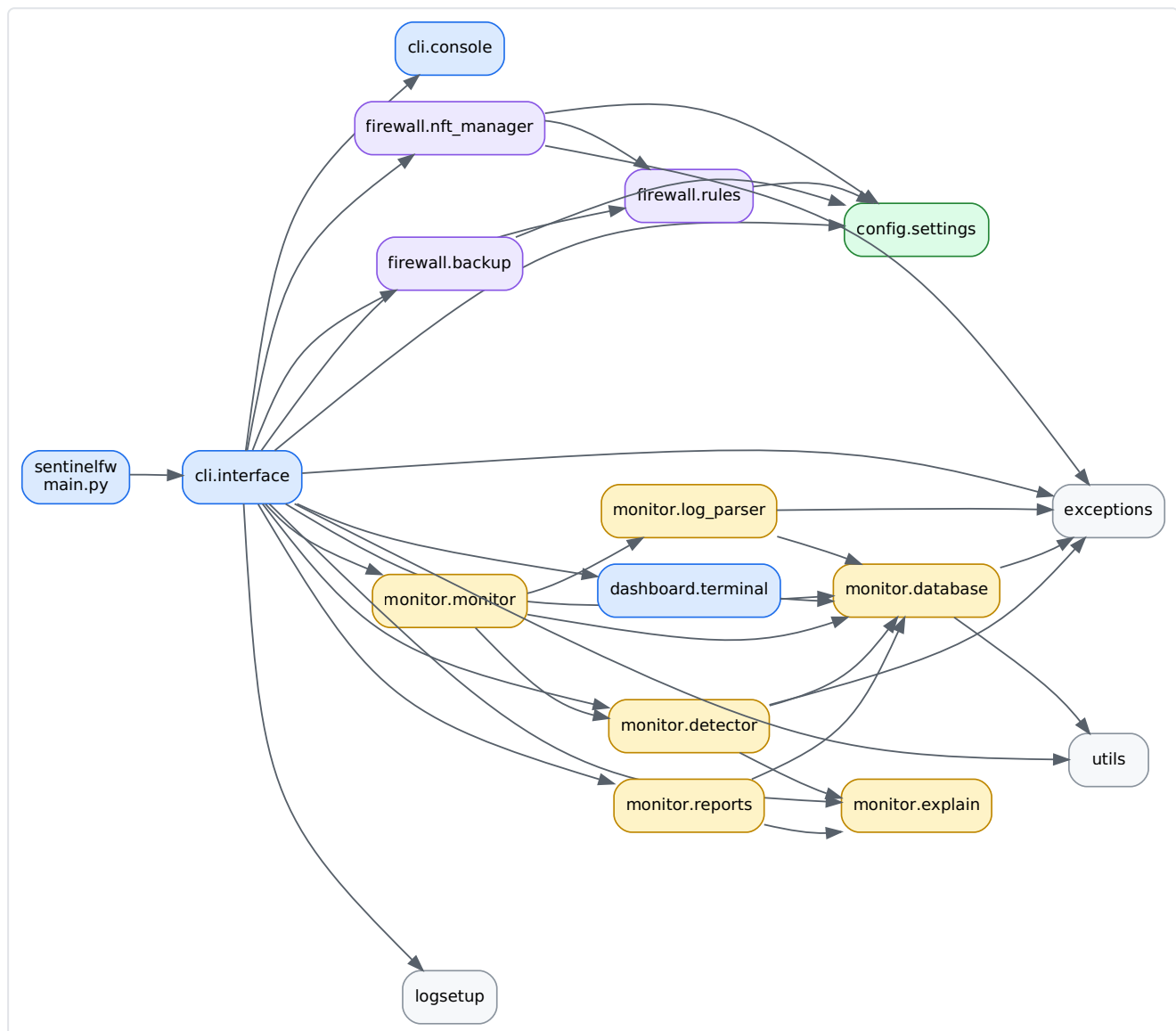


Figure 2 — Component and dependency diagram.

7.3 Data flow

There are two principal data flows.

Firewall flow. A command such as `firewall block-ip` validates input, checks for duplicates or contradictions, and appends a rule to `rules.yaml`. Nothing reaches the kernel. A later `firewall apply` renders the whole `rules.yaml` into one nft script, prints it, snapshots the live ruleset into `backups/`, confirms, runs `nft -c -f -` to check syntax, runs `nft -f -` to apply atomically, and re-reads the live ruleset to verify.

Monitoring flow. A log source (journal, file, stdin or demo) yields raw lines. `NFLogParser` recognises the `SENTINELFW-DROP` / `SENTINELFW-ACCEPT` prefixes and builds `FirewallEvent` objects, which are batched and bulk-inserted into SQLite. The `DetectionEngine` then runs five detectors as SQL aggregate queries over a sliding window, de-duplicates findings by fingerprint, applies a cooldown, and stores or updates alerts. Reports, the dashboard and `explain` read the resulting database.

8. UML and Design Diagrams

8.1 Use case view

The operator interacts with the rule, monitoring, alerting, reporting and explanation use cases. Three use cases cross a privilege boundary: applying rules to the kernel, backing up or restoring the ruleset, and reading the kernel journal all require root or group membership. Applying is the only use case that writes to the nftables kernel state.



Figure 3 — Use case diagram.

| 8.2 `firewall apply` control flow

The most safety-critical workflow is applying rules. Figure 4 shows every gate a run must pass. Each failure path leaves the machine unchanged, and the successful path always ends with a verification read-back rather than an assumption.

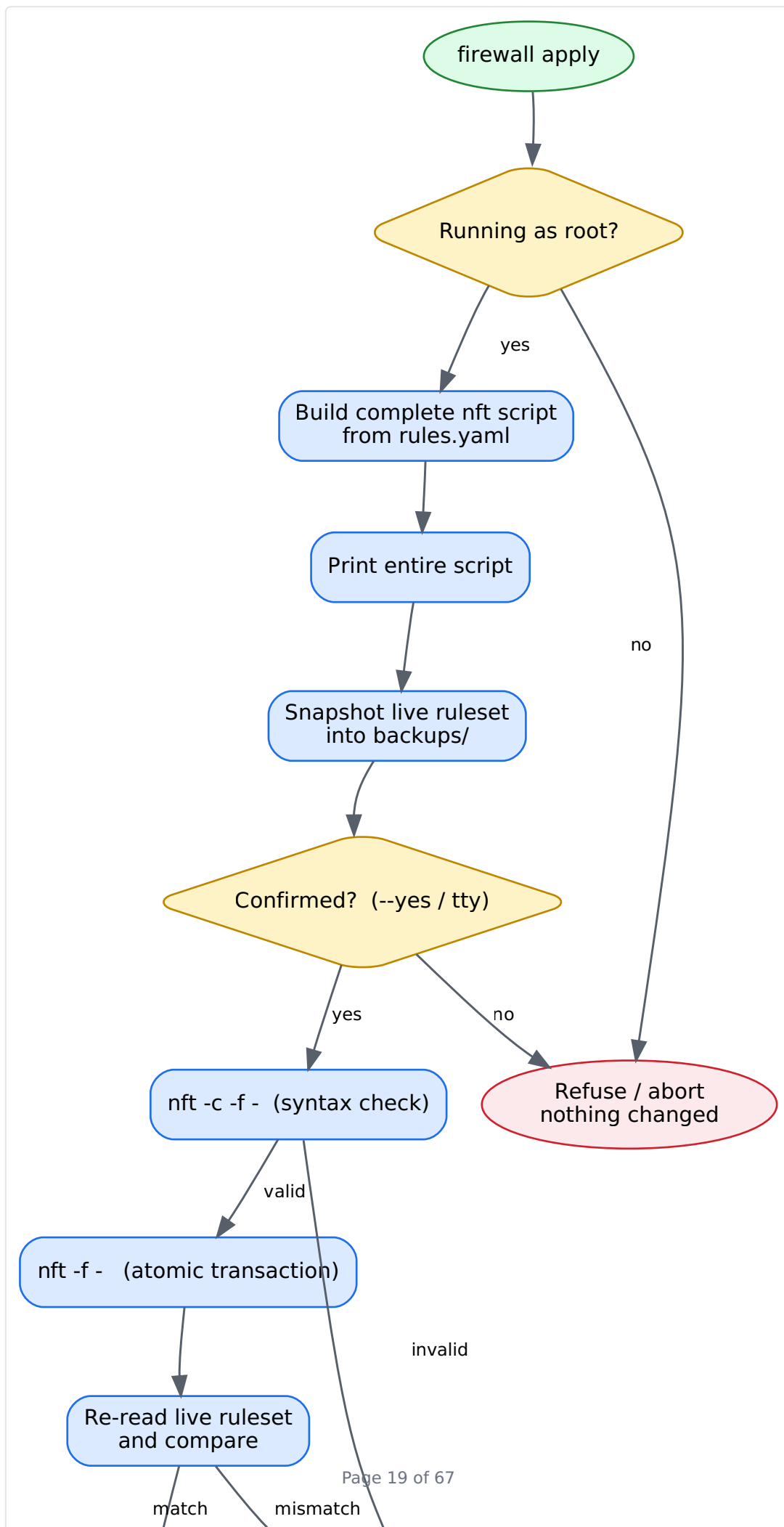


Figure 4 — `firewall apply` control flow.

8.3 Detection sequence

Figure 5 shows one monitoring cycle as a sequence: the collector opens a source, parses lines, batch-writes events, then hands control to the detection engine, which issues aggregate queries and records de-duplicated alerts before returning a result the collector can print.

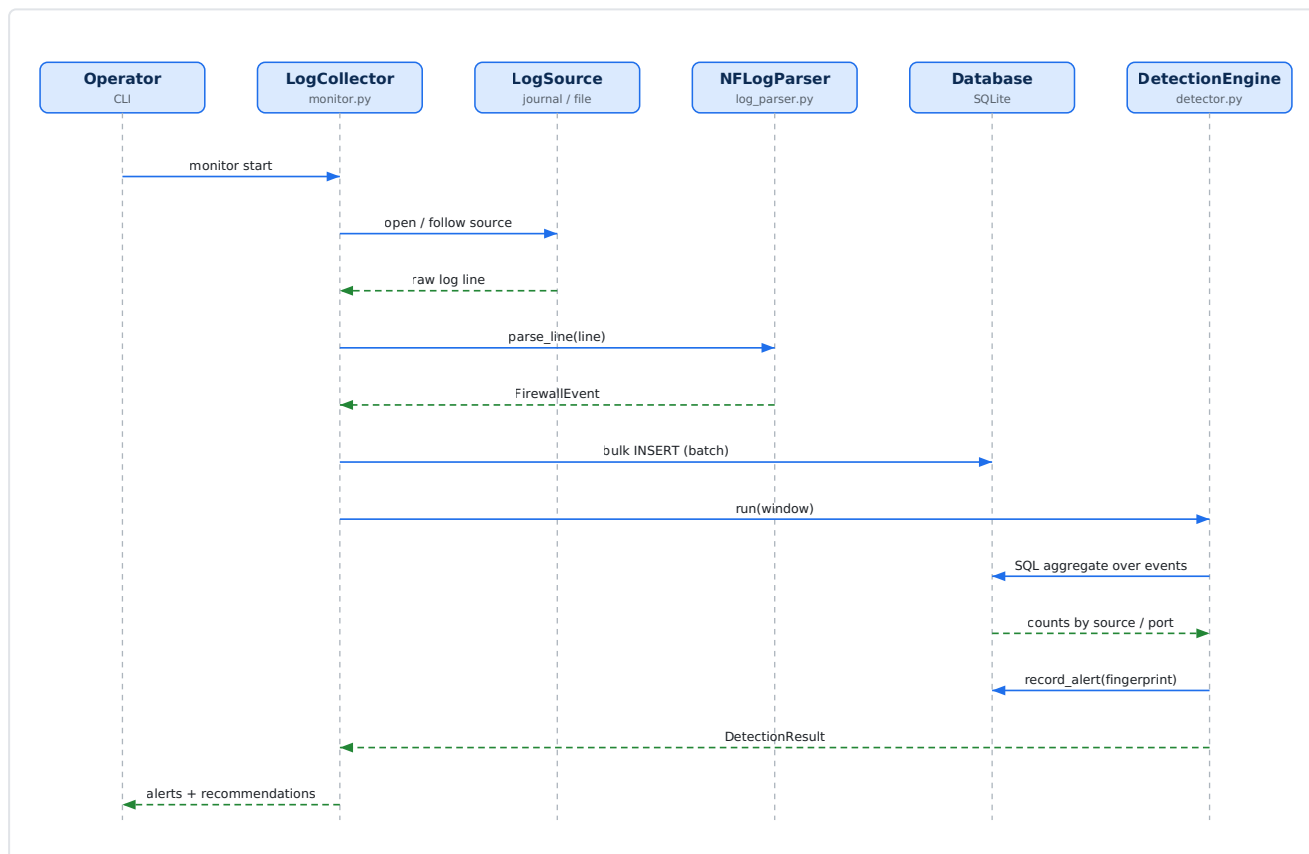


Figure 5 — Detection sequence diagram.

8.4 Monitoring activity

Figure 6 shows the monitoring loop as an activity diagram, including the batch flush decision and the two clean exits (source unavailable, and stop/Ctrl-C).

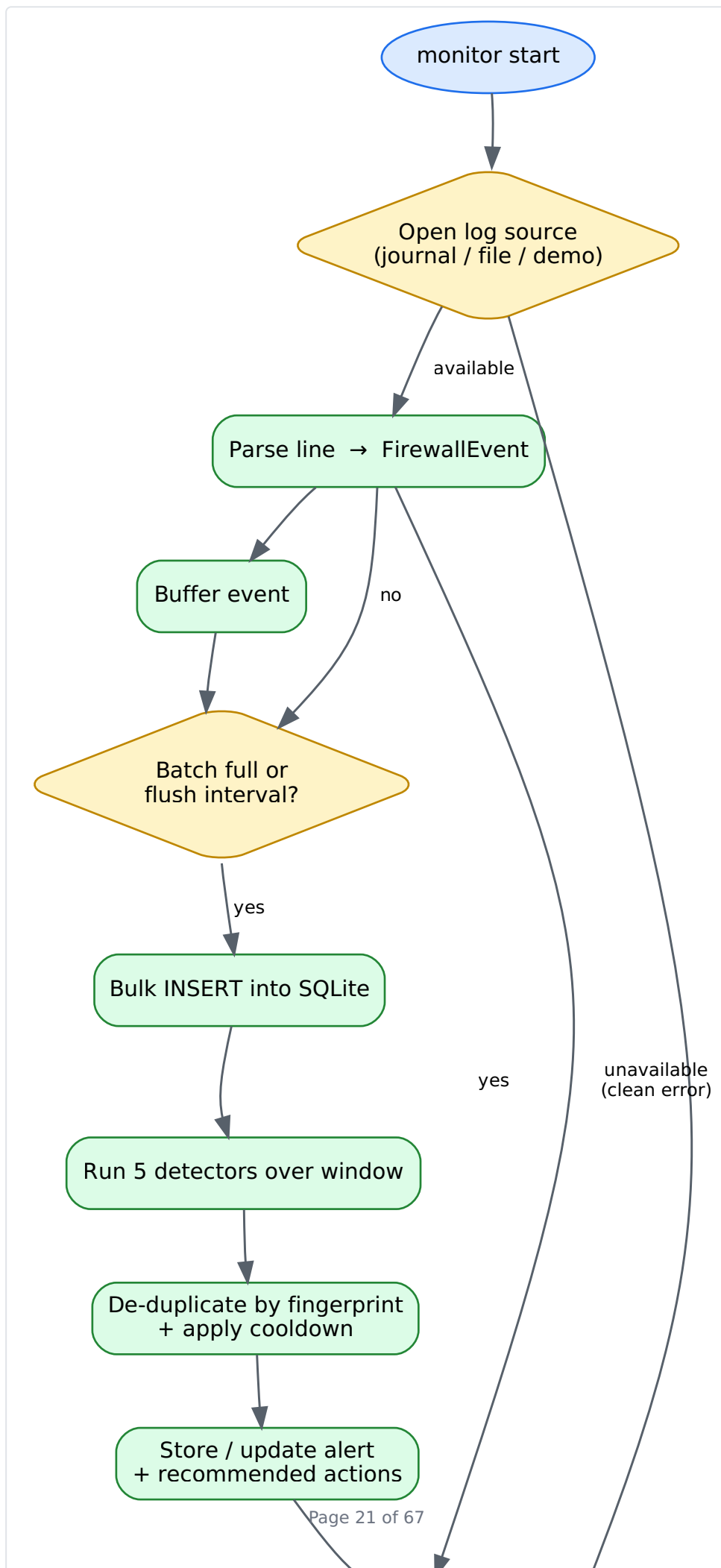


Figure 6 — Monitoring activity diagram.

8.5 Deployment view

SentinelFW runs as an unprivileged user for everything except applying, validating, restoring and journal reads. Figure 7 shows the deployment: the CLI and its project state live in the user session; `nft` and the SQLite database are touched on behalf of the user, with the root-only operations isolated.

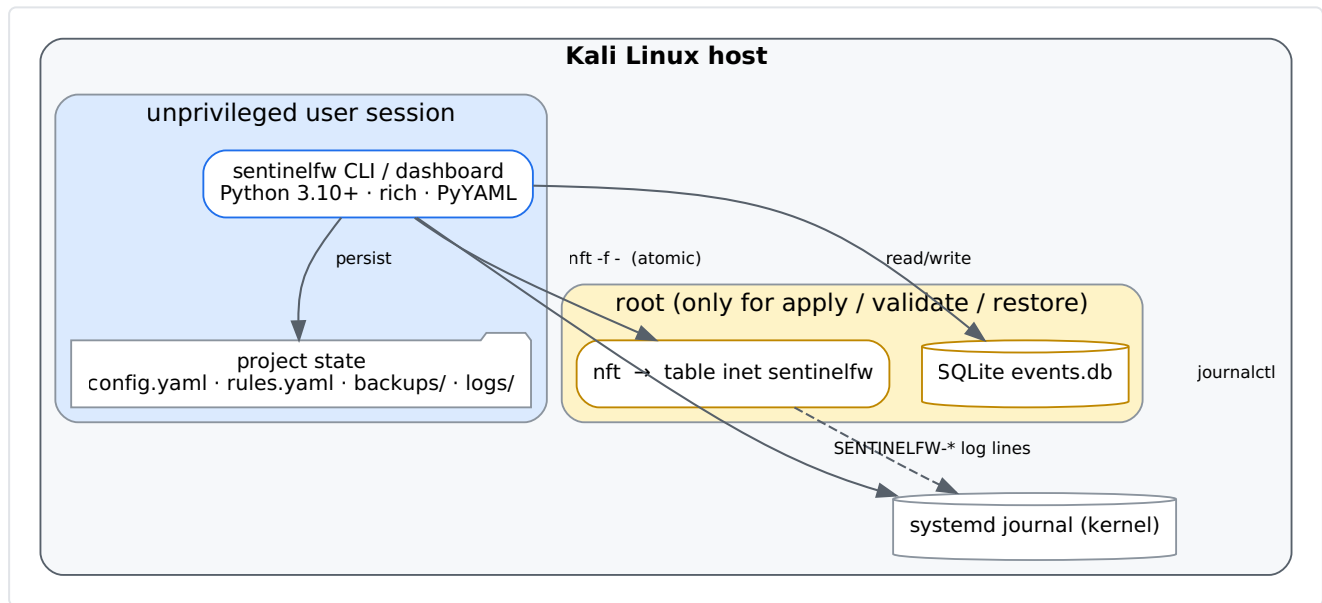


Figure 7 — Deployment diagram.

9. Database Design

SentinelFW stores monitoring state in a single SQLite file, by default `database/events.db`. The schema is versioned through a `meta` table and created on first use; the current schema version is `1`. The connection enables WAL mode, `NORMAL` synchronisation, a busy timeout, and foreign keys, and is guarded by a re-entrant lock so the collector and the CLI can coexist.

9.1 Entity-relationship diagram

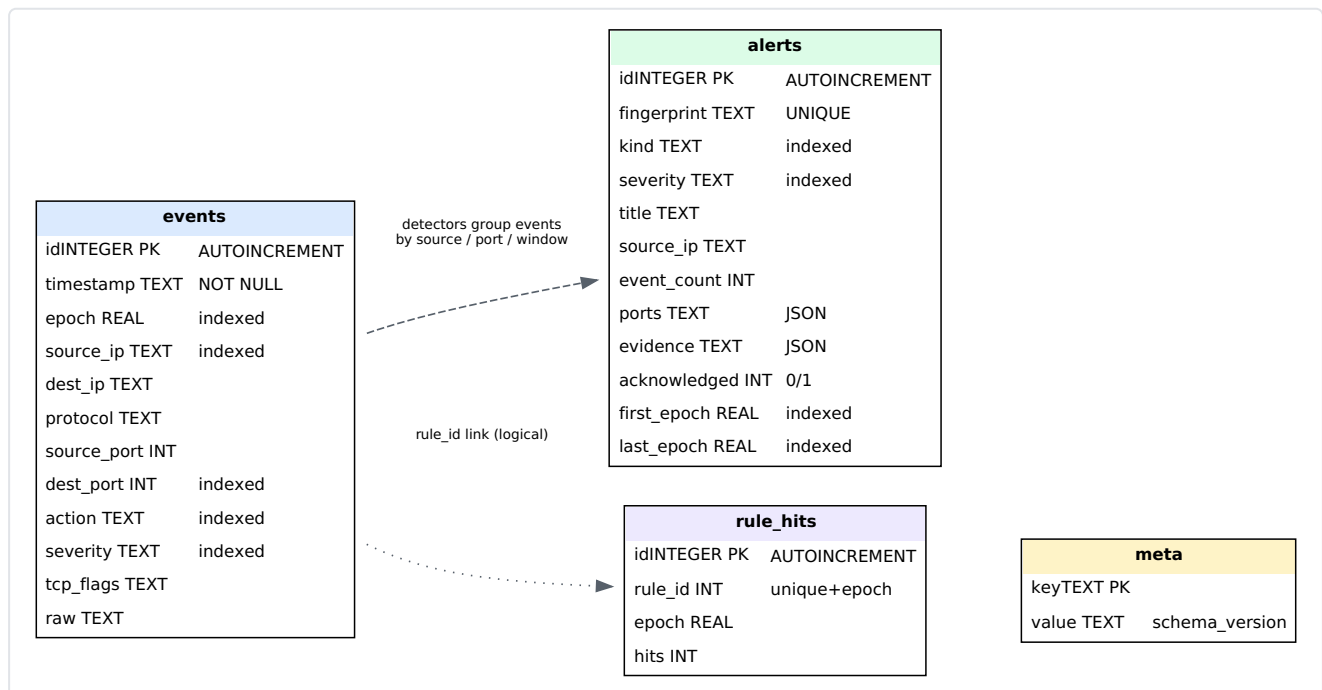


Figure 8 — Database entity-relationship diagram.

9.2 Tables

events stores one row per parsed firewall log line: timestamp and epoch, source and destination addresses and ports, protocol, action (`DROP` / `ACCEPT`), severity, TCP flags, packet length, and the raw line. It is indexed on epoch, `(source_ip, epoch)`, `(dest_port, epoch)`, `(action, epoch)` and `(severity, epoch)` so that detectors can answer windowed questions by index rather than by scanning the whole history.

alerts stores one row per distinct finding, keyed by a unique `fingerprint`. Repeated detections of the same finding update `event_count` and `last_seen` rather than inserting duplicates. The `ports` and `evidence` columns hold JSON. An `acknowledged` flag records that an operator has handled it, and `first_epoch` / `last_epoch` are indexed for time-range queries.

rule_hits accumulates per-rule hit counts in `(rule_id, epoch)` buckets using a unique index that lets repeated updates collapse into one row per second-window.

meta is a simple key/value store holding `schema_version` and `created_at`, and is the anchor for future migrations.

9.3 Retention

Events older than `database.retention_days` (90 by default) can be removed with `monitor prune`, and the file compacted with `monitor vacuum`. Pruning removes only events older than the cutoff; alerts are retained unless the whole database is reset.

10. Project Folder Structure

SentinelFW/	
├─ sentinelfw	executable launcher (runs with no install)
├─ main.py	entry point: python3 main.py
├─ cli/	
│ └─ interface.py	argument parsing, dispatch, confirmation flow
│ └─ console.py	previews, tables, confirmations
├─ config/	
│ └─ settings.py	typed configuration, validation, safe loading
├─ firewall/	
│ └─ rules.py	rule model, validation, nft rendering, rules.yaml
│ └─ nft_manager.py	the only code that runs nft; containment rules
│ └─ backup.py	snapshot and restore
├─ monitor/	
│ └─ database.py	SQLite schema, event and alert storage
│ └─ log_parser.py	kernel log parsing and log sources
│ └─ detector.py	the five detectors
│ └─ monitor.py	the collector that joins it all together
│ └─ explain.py	the knowledge base behind `explain`
│ └─ reports.py	text, markdown and JSON reports
├─ dashboard/	
│ └─ terminal.py	Rich dashboard
├─ exceptions.py	one exception per anticipated failure, each with a hint
├─ logsetup.py	rotating logs, including every nft command
├─ utils.py	time, formatting and small helpers
├─ version.py	version and schema constants
├─ tests/	160 tests; none of them touch the kernel
├─ docs/	SAFETY.md, this report, diagrams, screenshots
├─ pyproject.toml	packaging and the `sentinelfw` entry point
├─ requirements.txt	runtime dependencies
└─ README.md	user-facing overview

State that is *not* source — `config.yaml`, `rules.yaml`, `database/`, `logs/`, `backups/` and `reports/` — is gitignored and created next to the configuration file, so an installed `sentinelfw` run from a new directory creates its project there instead of scattering files back into the source tree.

11. Module Description

Table 3 — Module responsibilities.

Module	Responsibility
<code>cli/interface.py</code>	Argument parsing (groups, subcommands, global flags), command dispatch, exit-code mapping, top-level error handling
<code>cli/console.py</code>	Change previews, Rich tables, confirmation prompts, and the rule that a mutation must not assume consent
<code>config/settings.py</code>	Typed configuration sections, validation, <code>safe_load</code> , path resolution, file-permission hardening
<code>firewall/rules.py</code>	Rule model, value validation, nft rendering, ordering, duplicate/contradiction detection, <code>rules.yaml</code> persistence
<code>firewall/nft_manager.py</code>	The only code that runs <code>nft</code> ; builds the full script, enforces table containment, applies atomically, verifies
<code>firewall/backup.py</code>	Snapshot the live ruleset, list backups, restore (with an explicit destructive warning)
<code>monitor/log_parser.py</code>	Log sources (journal, file, stdin, demo, command) and parsing of <code>SENTINELFW-*</code> lines into <code>FirewallEvent</code>
<code>monitor/database.py</code>	SQLite schema and migrations, event/alert storage, aggregate queries, stats, prune, vacuum, export
<code>monitor/detector.py</code>	Five detectors, fingerprinting, cooldown, evidence and recommendation assembly
<code>monitor/monitor.py</code>	The <code>LogCollector</code> that reads a source, batches writes, runs detection, and reports statistics
<code>monitor/explain.py</code>	Knowledge base for ports, attack kinds, and remediation steps
<code>monitor/reports.py</code>	<code>SecurityReport</code> assembly and rendering to text, Markdown and JSON
<code>dashboard/terminal.py</code>	Responsive Rich dashboard with a <code>--once</code> snapshot mode
<code>exceptions.py</code>	Exception hierarchy with one class per anticipated failure, each carrying a hint and an exit code
<code>logsetup.py</code>	Rotating file logging, including every <code>nft</code> command the tool runs
<code>utils.py</code>	Time conversion, IP masking, human formatting, safe subprocess execution

11.1 The detector set

Table 5 — Detectors and default thresholds.

Kind	Default trigger	Window	Meaning
<code>port_scan</code>	20 distinct ports from one source	60 s	Reconnaissance across many services
<code>brute_force</code>	100 attempts on one port	300 s	Automated password or key probing
<code>repeated_block</code>	50 hits on one port	600 s	Persistent hammering of a closed port
<code>connection_spike</code>	300 connections overall	60 s	Unusual aggregate volume

Kind	Default trigger	Window	Meaning
<code>sensitive_port</code>	Any traffic to a watched port	60 s	Traffic to services that should not be exposed

Every threshold and window lives under `detection:` in `config.yaml`, together with `alert_cooldown_seconds` (900 by default). The watched ports are FTP (21), SSH (22), Telnet (23), SMTP (25), DNS (53), HTTP (80), POP3 (110), IMAP (143), HTTPS (443), SMB (445), MSSQL (1433), MySQL (3306), RDP (3389), PostgreSQL (5432), VNC (5900), Redis (6379), HTTP-alt (8080/8443) and MongoDB (27017).

12. Implementation Details

12.1 The safety invariants

The heart of the project is a small set of invariants, each enforced in code and covered by tests. They are stated in full in `docs/SAFETY.md`.

Table 8 — Safety invariants and enforcement.

#	Invariant	Enforced by
1	A rule change never touches the kernel	<code>firewall/rules.py</code> has no access to <code>subprocess</code> ; only <code>nft_manager.py</code> runs <code>nft</code> ; <code>apply</code> is explicit
2	SentinelFW owns exactly one <code>nftables</code> object	<code>nft_manager.py</code> refuses any table other than <code>inet sentinelfw</code> ; tests assert no other table statement appears
3	A failed apply cannot leave a broken chain	Script emits <code>flush chain</code> , not <code>flush table</code> ; a single <code>nft -f -</code> transaction; verify by read-back
4	Consent is never assumed	<code>cli/console.py</code> refuses a confirmation when non-interactive and <code>--yes</code> is absent
5	Input cannot escape into the script	IPs via <code>ipaddress</code> , ports range-checked, protocols whitelisted, comments sanitised, custom rules reject newline/;
6	Restore is loud about being destructive	<code>firewall/backup.py</code> plus explicit CLI warning; no quieter flag exists
7	Configuration is data, never code	<code>yaml.safe_load</code> only, everywhere
8	Nothing runs a shell	<code>utils.run_command</code> uses argument lists; no <code>shell=True</code> anywhere

12.2 Applying rules

`firewall apply` is the only command that changes the kernel, and it is deliberately ceremonial:

1. Refuse unless running as root.
2. Build the complete script from `rules.yaml` and print all of it.
3. Snapshot the live ruleset into `backups/`.
4. Ask for confirmation, unless `--yes`.
5. Re-check the script with `nft -c -f -` (check only).
6. Apply it with `nft -f -` as one atomic transaction.
7. Read the live ruleset back and compare it with the intent.
8. Log every command that was run.

If any of steps 6–8 fails, the run is reported as a failure. The tool never claims to have installed rules it has not verified.

12.3 Rule ordering and conflicts

Rules are rendered in an order that evaluates allows before drops so that an explicit allow is not shadowed by a broader block. When a newly added rule would make an existing rule unreachable, SentinelFW says so. Recording an identical or contradicting rule is refused; `--`

`yes` suppresses prompts only, and does **not** bypass that check — overriding it is the separate, explicit `--force` flag. This distinction matters because an unattended run must not quietly accumulate duplicate rules.

12.4 Parsing without crashing

The parser is written to be defensive: unrelated kernel lines, empty lines and missing fields all return `None` or a partial event rather than raising. TCP flags written by nft as `syn,ack` are condensed into a short form. Severity is assigned at parse time based on whether the destination port is watched.

13. Configuration

`config.yaml` is created with mode `0600` and commented defaults. Relative paths resolve next to the configuration file, so pointing `--config` or `SENTINELFW_CONFIG` at another directory moves the whole project there.

Table 4 — Principal configuration defaults.

Key	Default	Meaning
<code>firewall.table</code>	<code>sentinelfw</code>	The one table SentinelFW owns
<code>firewall.family</code>	<code>inet</code>	Address family
<code>firewall.chain</code>	<code>input</code>	Chain name
<code>firewall.priority</code>	<code>-10</code>	Base chain priority (before most services)
<code>firewall.policy</code>	<code>accept</code>	Chain policy; cannot lock the host out
<code>firewall.log_prefix</code>	<code>SENTINELFW</code>	Log prefix the parser looks for
<code>database.path</code>	<code>database/events.db</code>	SQLite file
<code>database.retention_days</code>	<code>90</code>	Retention window
<code>monitoring.source</code>	<code>journal</code>	<code>journal</code> , <code>file</code> , <code>stdin</code> or <code>demo</code>
<code>detection.port_scan_threshold</code>	<code>20</code>	Distinct ports per source
<code>detection.brute_force_threshold</code>	<code>100</code>	Attempts on one port
<code>detection.alert_cooldown_seconds</code>	<code>900</code>	Suppression window per finding
<code>report.default_period</code>	<code>24h</code>	Default reporting window
<code>dashboard.refresh_seconds</code>	<code>3.0</code>	Live dashboard refresh

14. Interface Design

SentinelFW is a text-first tool. It uses Rich to render tables, panels and a live dashboard, and it adapts to terminal width: on a narrow window it drops optional columns rather than crushing every column to one character. Machine-readable commands support `--json` and always emit a single valid JSON document, so output can be piped to `jq`.

14.1 Environment check

`doctor` reports the state of the machine — Python version, packages, the `nft` binary, privileges, configuration permissions, journal access, the firewall table, the database and free disk — and exits with code 6 when something needs attention.

```
lucky@sentinelfw:~/SentinelFW$ ./sentinelfw doctor
```

Check	Result	SentinelFW doctor Detail
Python 3.14.7	OK	supported (3.10+)
package: rich	OK	installed
package: pyyaml	OK	installed
nft binary	OK	nftables v1.1.7 (Commodore Bullmoose #8)
root privileges	PROBLEM	no - reading the live ruleset and applying changes need sudo
config.yaml	OK	/tmp/opencode/sfwlab/config.yaml
config permissions	OK	0600
rules.yaml	OK	no rules file yet (fine)
firewall table	PROBLEM	not installed - run: sudo sentinelfw firewall apply
journalctl	OK	available
journal access	OK	can read the kernel journal
event database	OK	will be created at /tmp/opencode/sfwlab/database/events.db
log file	OK	/tmp/opencode/sfwlab/logs/sentinelfw.log
free disk	OK	3800 MiB free in /tmp/opencode/sfwlab

2 check(s) need attention:

- root privileges
- firewall table

Figure 12 — `sentinelfw doctor` environment check (non-root, firewall not yet installed).

14.2 Recording and previewing a rule

Recording a rule shows what will happen and what the `nft` statement will be, while making clear that nothing has been applied yet.

```
lucky@sentinelFW:~/SentinelFW$ ./sentinelFW firewall block-ip 203.0.113.7 -r scanner -y
```

```

FIREWALL CHANGE REQUESTED
Action      : Block IP address
Target      : 203.0.113.7
Effect      : Inbound packets from 203.0.113.7 are dropped before they reach
any service.
Backend     : nftables (requires root)
Reason      : scanner
Commands    :
    stored in /tmp/opencode/sfwlab/rules.yaml
    (not applied to the kernel yet)

    [after 'firewall apply']
    add rule inet sentinelFW input ip saddr 203.0.113.7 counter log prefix
    "SENTINELFW-DROP " level info drop comment "scanner"

Undo        : sentinelFW firewall remove <id>   (after it is created)

```

```

--yes given; proceeding without confirmation
OK Rule #1 saved to /tmp/opencode/sfwlab/rules.yaml
Not active in the kernel yet. Install it with:
sudo sentinelFW firewall apply

```

Figure 13 — Recording a block rule; the kernel is not touched.

```
lucky@sentinelFW:~/SentinelFW$ ./sentinelFW firewall list
```

SentinelFW rules (3)

ID	Action	Kind	Target	Comment	Enabled
3	ACCEPT	ip allow	198.51.100.0/24	lab net	yes
1	DROP	ip block	203.0.113.7	scanner	yes
2	DROP	port block	tcp/23 (telnet)	telnet off	yes

```

Use --live to compare against the rules loaded in the kernel.
These rules are stored in /tmp/opencode/sfwlab/rules.yaml.
They affect the kernel only after: sudo sentinelFW firewall apply

```

Figure 14 — Listing stored rules from `rules.yaml`.

```
lucky@sentinelFW:~/SentinelFW$ ./sentinelFW firewall preview

nft script preview
Table      : inet sentinelFW
Chain      : inet sentinelFW input
Rules      : 3 stored, 3 enabled

This is the exact script that 'firewall apply' feeds to 'nft -f -' in a
single atomic transaction:

add table inet sentinelFW
add chain inet sentinelFW input { type filter hook input priority -10;
policy accept; }
flush chain inet sentinelFW input
add rule inet sentinelFW input ip saddr 198.51.100.0/24 counter log prefix
"SENTINELFW-ACCEPT " level info accept comment "lab net"
add rule inet sentinelFW input ip saddr 203.0.113.7 counter log prefix
"SENTINELFW-DROP " level info drop comment "scanner"
add rule inet sentinelFW input tcp dport 23 counter log prefix
"SENTINELFW-DROP " level info drop comment "telnet off"

Nothing above has been executed. To install it:
sudo sentinelFW firewall apply
█
```

Figure 15 — Previewing the exact `nft` script that `apply` would run.

14.3 Monitoring and detection

The demo stream lets a learner exercise the whole pipeline without root, a real attacker, or real traffic. It is labelled as synthetic everywhere it appears.

```

lucky@sentinelfw:~/SentinelFW$ ./sentinelfw monitor demo --seconds 8 -y

  Demo mode
Source   : synthetic demo traffic
Duration : 8s
Database : /tmp/opencode/sfwlab/database/events.db

Demo data is fabricated. It is written to the same database as real events
and is labelled 'demo' in every explanation.
Use 'monitor reset' to clear it afterwards.

--yes given; proceeding without confirmation
  ALERT
HIGH Probed 3 sensitive service(s) (3 blocked attempts)
source 66.240.192.10
66.240.192.10 was blocked 3 time(s) while targeting 3 service(s) that
should not be reachable from ...
These services have no business facing the open internet. Any inbound
attempt is either misconfigur...
-> sentinelfw firewall block-ip 66.240.192.10 # then: firewall apply
-> sentinelfw firewall block-port 22 --protocol tcp

  ALERT
HIGH Probed 11 sensitive service(s) (28 blocked attempts)
source 45.33.22.1
45.33.22.1 was blocked 28 time(s) while targeting 11 service(s) that
should not be reachable from a...
These services have no business facing the open internet. Any inbound
attempt is either misconfigur...
-> sentinelfw firewall block-ip 45.33.22.1 # then: firewall apply
-> sentinelfw firewall block-port 21 --protocol tcp

  ALERT
HIGH Probed 1 sensitive service(s) (26 blocked attempts)
source 103.21.244.7
103.21.244.7 was blocked 26 time(s) while targeting 1 service(s) that
should not be reachable from ...
These services have no business facing the open internet. Any inbound
attempt is either misconfigur...
-> sentinelfw firewall block-ip 103.21.244.7 # then: firewall apply
-> sentinelfw firewall block-port 22 --protocol tcp

  Monitor summary
Lines read      : 67
Events stored   : 67
Batch writes    : 1
Detection cycles : 1
Last cycle      : {"scanned_events": 67, "alerts": 3, "new_alerts": 3,
"updated_alerts": 0, "duration_seconds": 0.003, "kinds": ["sensitive_port"]}
Errors          : none
Last event      : 2026-10-10T06:18:43+00:00

3 new alert(s) raised. Review: sentinelfw alerts list

```

Figure 16 — Synthetic traffic demonstration, with alerts raised.

```
lucky@sentinelFW:~/SentinelFW$ ./sentinelFW dashboard --once
```

SentinelFW v1.0.0 local firewall security assistant
last 24h | generated 2026-10-10T06:18:59Z

NOT APPLIED
run: firewall apply

3
tracked

67
last 24h

3 (3 high)
open

6
unique IPs

05:18 ■ 67 (67 blocked)

Top blocked sources

Source	Attempts	Ports	Last seen
45.33.22.1	35	15	06:18:36
103.21.244.7	26	1	06:18:36
66.240.192.10	3	3	06:18:43
198.51.100.7	1	1	06:18:40
91.243.44.12	1	1	06:18:38
185.220.101.4	1	1	06:18:36

Most targeted ports

Port	Service	Attempts
22	ssh	28
80	http	6
21	ftp	4
443	https	4
25	smtp	3
3389	rdp	3
27017	mongodb	3
23	telnet	2

Recent alerts

HIGH Probed 3 sensitive service(s) (3 blocked attempts)
66.240.192.10 was blocked 3 time(s) while targeting 3 service(s) that should not be reachable from an untrusted network: 22 (ssh), 53 (domain), 80 (http).
source: 66.240.192.10 port: 22 last: 2026-10-10T06:18:43Z

HIGH Probed 1 sensitive service(s) (26 blocked attempts)
103.21.244.7 was blocked 26 time(s) while targeting 1 service(s) that should not be reachable from an untrusted network: 22 (ssh).
source: 103.21.244.7 port: 22 last: 2026-10-10T06:18:36Z

HIGH Probed 11 sensitive service(s) (28 blocked attempts)
45.33.22.1 was blocked 28 time(s) while targeting 11 service(s) that should not be reachable from an untrusted network: 21 (ftp), 23 (telnet), 25 (smtp), 53 (domain), 80 (http), 443 (https) and 5 more.
source: 45.33.22.1 port: 21 last: 2026-10-10T06:18:36Z

Latest events

Time	Action	Source	Port
06:18:43	DROP	66.240.xx.xx	80
06:18:42	DROP	66.240.xx.xx	22
06:18:40	DROP	198.51.xx.xx	80
06:18:39	DROP	66.240.xx.xx	53
06:18:38	DROP	91.243.xx.xx	22
06:18:36	DROP	185.220.xx.xx	80
06:18:36	DROP	103.21.xx.xx	22
06:18:36	DROP	103.21.xx.xx	22
06:18:36	DROP	103.21.xx.xx	22
06:18:36	DROP	103.21.xx.xx	22
06:18:36	DROP	103.21.xx.xx	22
06:18:36	DROP	103.21.xx.xx	22

Figure 17 — Terminal dashboard snapshot.

```

lucky@sentinelFW:~/SentinelFW$ ./sentinelFW alerts list --explain
Alerts (3)

```

ID	Severity	Finding	Source	Events	Last seen
1	HIGH	Probed 3 sensitive service(s) (3 blocked attempts)	66.240.192.10	3	2026-10-10T06:18:43
3	HIGH	Probed 1 sensitive service(s) (26 blocked attempts)	103.21.244.7	26	2026-10-10T06:18:36
2	HIGH	Probed 11 sensitive service(s) (28 blocked attempts)	45.33.22.1	28	2026-10-10T06:18:36

```

- Probed 3 sensitive service(s) (3 blocked attempts) -
66.240.192.10 was blocked 3 time(s) while targeting 3 service(s) that should
not be reachable from an untrusted network: 22 (ssh), 53 (domain), 80
(http).

Why this matters:
These services have no business facing the open internet. Any inbound
attempt is either misconfiguration or hostile intent.

What to do:
- sentinelFW firewall block-ip 66.240.192.10 # then: firewall apply
- sentinelFW firewall block-port 22 --protocol tcp
- Keep the block in place
- Bind the service to localhost or a private interface
- If remote access is required, reach it over a VPN or an SSH tunnel

- Probed 1 sensitive service(s) (26 blocked attempts) -
103.21.244.7 was blocked 26 time(s) while targeting 1 service(s) that should
not be reachable from an untrusted network: 22 (ssh).

Why this matters:
These services have no business facing the open internet. Any inbound
attempt is either misconfiguration or hostile intent.

What to do:
- sentinelFW firewall block-ip 103.21.244.7 # then: firewall apply
- sentinelFW firewall block-port 22 --protocol tcp
- Keep the block in place
- Bind the service to localhost or a private interface
- If remote access is required, reach it over a VPN or an SSH tunnel

- Probed 11 sensitive service(s) (28 blocked attempts) -
45.33.22.1 was blocked 28 time(s) while targeting 11 service(s) that should
not be reachable from an untrusted network: 21 (ftp), 23 (telnet), 25
(smtp), 53 (domain), 80 (http), 443 (https) and 5 more.

Why this matters:
These services have no business facing the open internet. Any inbound
attempt is either misconfiguration or hostile intent.

What to do:
- sentinelFW firewall block-ip 45.33.22.1 # then: firewall apply
- sentinelFW firewall block-port 21 --protocol tcp
- Keep the block in place
- Bind the service to localhost or a private interface
- If remote access is required, reach it over a VPN or an SSH tunnel

Period: last 24h. Acknowledge with: sentinelFW alerts ack <id>

```

Figure 18 — Alerts with plain-language explanations and next steps.

```

lucky@sentinelFW:~/SentinelFW$ ./sentinelFW report generate
=====
                        SentinelFW Security Report
=====
Generated : 2026-10-10T06:19:28Z
Period    : last 24h
Window    : 2026-10-09T06:19:28Z -> 2026-10-10T06:19:28Z

-- Summary -----
Events recorded      : 67
Blocked / rejected  : 67
Accepted            : 0
Unique sources      : 6
Unique ports targeted: 16
Alerts raised       : 3
  - high            : 3

-- Top attack sources -----
Source              Attempts  Ports
45.33.22.1          35       15
103.21.244.7        26       1
66.240.192.10       3        3
198.51.100.7        1        1
91.243.44.12        1        1
185.220.101.4       1        1

-- Most targeted ports -----
Port  Service      Proto  Attempts
22    ssh           tcp    28
80    http          tcp    6
21    ftp           tcp    4
443   https          tcp    4
25    smtp          tcp    3
3389  rdp             tcp    3
27017 mongodb        tcp    3
23    telnet        tcp    2
53    dns           tcp    2
111   sunrpc         tcp    2

-- Threats detected -----
[HIGH] ] Probed 3 sensitive service(s) (3 blocked attempts)
        ] 66.240.192.10 was blocked 3 time(s) while targeting 3 ser..
[HIGH] ] Probed 1 sensitive service(s) (26 blocked attempts)
        ] 103.21.244.7 was blocked 26 time(s) while targeting 1 ser..
[HIGH] ] Probed 11 sensitive service(s) (28 blocked attempts)
        ] 45.33.22.1 was blocked 28 time(s) while targeting 11 serv..

-- Firewall state -----
Backend : nftables v1.1.7 (Commodore Bullmoose #8)
Table   : inet sentinelFW (not installed)
Rules   : 3 tracked
        #3 ACCEPT ip allow 198.51.100.0/24
        #1 DROP  ip block  203.0.113.7
        #2 DROP  port block tcp/23 (telnet)

-- Recommendations -----
1. sentinelFW firewall block-ip 66.240.192.10 # then: firewall apply
2. sentinelFW firewall block-port 22 --protocol tcp
3. Keep the block in place
4. Bind the service to localhost or a private interface
5. If remote access is required, reach it over a VPN or an SSH tunnel
6. sentinelFW firewall block-ip 103.21.244.7 # then: firewall apply
7. sentinelFW firewall block-ip 45.33.22.1 # then: firewall apply
8. sentinelFW firewall block-port 21 --protocol tcp
9. sentinelFW firewall block-port 22 # then: sentinelFW firewall apply
10. sentinelFW firewall block-port 21 # then: sentinelFW firewall apply
11. SentinelFW's nftables table is not installed. Apply your rules with:
    sentinelFW firewall apply

-- Notes -----
- Severity is derived from event volume and pattern heuristics, not from
  packet inspection. Treat findings as leads to investigate, not proof of
  compromise.
- Database size: 92 KiB at /tmp/opencode/sfwlab/database/events.db.

=====
Report generated locally by SentinelFW. No data left this machine.
=====

Save it with: sentinelFW report generate --save --format text

```

Figure 19 — Generated security report (text format).

```
lucky@sentinelfw:~/SentinelFW$ ./sentinelfw explain port 22
```

Port 22

Port	: 22
Service	: ssh
Risk	: HIGH

Remote administration and the single most attacked service on the internet.

Why attackers care:
Expect constant password and key-probing attempts on any public IP.

Suggestion: `sentinelfw firewall block-port 22`

Figure 20 — `explain port 22`, from the knowledge base.

15. Command Reference

Table 7 — Command reference (abridged).

Group	Command	Purpose
firewall	<code>block-ip</code> , <code>allow-ip ADDRESS</code>	Store a drop/accept rule for an address or network
firewall	<code>block-port</code> , <code>allow-port PORT</code>	Store a drop/accept rule for a TCP/UDP port
firewall	<code>list</code> (<code>--live</code>)	Show stored rules; optionally diff against the kernel
firewall	<code>status</code>	Backend availability and rule sync
firewall	<code>preview</code>	Print the exact nft script; execute nothing
firewall	<code>validate</code>	Ask <code>nft</code> to check the script (root)
firewall	<code>apply</code>	Install rules atomically (root)
firewall	<code>remove</code> , <code>enable</code> , <code>disable ID</code>	Manage stored rules
firewall	<code>backup</code> , <code>backups</code> , <code>restore FILE</code>	Snapshot and restore (root)
firewall	<code>flush</code>	Remove the whole SentinelFW table (root)
firewall	<code>enable-logging</code> (<code>--off</code>)	Emit <code>SENTINELFW-*</code> log lines
monitor	<code>start</code> , <code>poll</code> , <code>ingest FILE</code> , <code>demo</code>	Collect logs
monitor	<code>status</code> , <code>detect</code> (<code>--dry-run</code>)	Inspect and run detection
monitor	<code>prune</code> , <code>vacuum</code> , <code>reset</code>	Retention and maintenance
alerts	<code>list</code> , <code>show ID</code> , <code>ack ID</code>	Inspect and acknowledge findings
report	<code>generate</code> (<code>--format</code> , <code>--brief</code> , <code>--save</code>), <code>list</code>	Build and list reports
dashboard	(none) / <code>--once</code>	Live view or a single snapshot
explain	<code>port PORT</code> , <code>attack KIND</code> , <code>event</code>	Knowledge base lookups
db	<code>stats</code> , <code>export</code> (<code>--format</code> , <code>--output</code>)	Database inspection and export
config	<code>init</code> , <code>show</code> , <code>path</code> , <code>check</code> , <code>harden</code>	Configuration management
doctor	(none)	Environment and permission checks

Global flags — `--config`, `--dry-run`, `-y/--yes`, `--no-color`, `--json`, `-v/--verbose`, `--debug` — are accepted before or after the command, which is what people actually type.

16. Exit Codes

Structured exit codes let scripts branch on the reason for a failure without parsing prose.

Table 6 — Exit codes.

Code	Meaning	Raised by
0	Success	—
1	Operational error	<code>FirewallError</code> , <code>DatabaseError</code> , <code>NftablesCommandError</code> , <code>ReportError</code> , <code>MonitoringError</code> , <code>BackupError</code>
2	Configuration file not found	<code>ConfigNotFoundError</code>
3	Insufficient privileges	<code>RootRequiredError</code> , <code>PrivilegeError</code> , <code>SafetyViolationError</code>
4	Validation failure	<code>ConfigValidationError</code> , <code>RuleValidationError</code> , <code>RuleConflictError</code>
5	Aborted by the operator	<code>UserAbortError</code> , including Ctrl-C
6	Dependency or environment problem	<code>NftablesNotAvailableError</code> , <code>LogSourceError</code>

Codes 2, 3, 4 and 6 mean "retrying with different inputs or privileges will help"; code 1 means something operational went wrong; code 5 means nothing was changed.

17. Test Plan

17.1 Strategy

The test strategy rests on one hard constraint: **the suite must never invoke `nft`**. A test that could change a live firewall is not a test, it is an incident. Consequently the suite runs entirely against temporary directories and verifies three things:

1. **Generation** — the `nft` script SentinelFW *would* run is correct, contained and injection-safe.
2. **Refusal** — every operation that requires root fails cleanly when it does not have it, changing nothing.
3. **Behaviour** — parsing, storage, detection, reporting, configuration and CLI dispatch behave as specified.

Root-only branches are therefore covered by asserting that they refuse, not by applying rules. This is a deliberate, documented trade-off: the suite proves the refusal paths and the script generation, but it cannot prove that a real `nft` accepts the generated syntax. That is exactly why `firewall validate` and `firewall preview` exist — they let a person check the script on a machine they control before anything is installed.

17.2 Levels of testing

- **Unit tests** — value validation, rendering, parsing, configuration coercion.
- **Integration tests** — rule store round-trips, database storage and queries, detector pipelines over stored events.
- **End-to-end tests** — the CLI invoked through `main(argv)` against a throwaway project, asserting exit codes and output.
- **Safety tests** — containment, injection, confirmation bypass, and the root-refusal paths.

17.3 Environment

Table 9 — Test environment.

Item	Value
Operating system	Kali Linux (Debian-based)
Python	CPython 3.14.7
pytest	9.1.1
Coverage plugin	pytest-cov
nftables (host)	v1.1.7
SQLite	3.53.4
Privileges during tests	Unprivileged user
Kernel interaction during tests	None
Test files	4
Test functions	160
Total suite runtime	≈ 51 seconds

17.4 Test data isolation

An autouse fixture points `SENTINELFW_CONFIG` at a temporary directory and changes the working directory into it, so no test can write to the operator's real `config.yaml`, `rules.yaml`, database or logs. The CLI tests patch the interactivity probe directly rather than replacing `sys.stdout`, which would break Rich's rendering rather than exercise the refusal path.

18. Test Cases and Results

All 160 tests were executed with `python3 -m pytest`. The result was **160 passed in 50.93 s**, with no failures, errors or skips. The tables in this section present the results grouped by concern. Because many tests are parameterised, a single row may represent several generated cases; the total across the tables is 160.

18.1 Rule validation and injection

Table 10 — Rule validation and injection test results.

ID	Test	Expected result	Actual result	Status
R-01	Valid IPv4/IPv6 address and network accepted (4 cases)	Normalised value returned	Normalised	Pass
R-02	Invalid IP values rejected (7 cases)	<code>RuleValidationError</code>	Raised	Pass
R-03	Absurdly broad networks (<code>0.0.0.0/0</code> , <code>10.0.0.0/4</code>) rejected	<code>RuleValidationError</code>	Raised	Pass
R-04	Valid ports accepted, trimmed (3 cases)	Integer returned	Integer	Pass
R-05	Invalid ports rejected (<code>0</code> , <code>65536</code> , <code>-1</code> , text, float)	<code>RuleValidationError</code>	Raised	Pass
R-06	Protocol normalisation (<code>TCP→tcp</code> , <code>all→any</code> , <code>None→any</code>)	Canonical value	Canonical	Pass
R-07	Unknown protocol (<code>sctp</code>) rejected	<code>RuleValidationError</code>	Raised	Pass
R-08	Port service name is best-effort (<code>22→ssh</code> , <code>None→""</code>)	Correct name	Correct	Pass
R-09	IP rule target and <code>is_ip_rule</code>	Correct target	Correct	Pass
R-10	Port rule target names the service	Target contains <code>22</code> and <code>ssh</code>	Correct	Pass
R-11	Unknown kind/action rejected	<code>RuleValidationError</code>	Raised	Pass
R-12	Custom rule rejects newline statement smuggling	<code>RuleValidationError</code>	Raised	Pass
R-13	Custom rule rejects <code>;</code> separator smuggling	<code>RuleValidationError</code>	Raised	Pass
R-14	Comment cannot break out of nft quotes	Even quote count; no <code>;</code> drop;	Safe	Pass
R-15	Comment newline cannot smuggle a second rule	No newline survives	Safe	Pass
R-16	Quotes and backslashes in comments sanitised	Balanced quotes	Safe	Pass
R-17	Rendered rule contains expected tokens	Tokens present	Present	Pass
R-18	<code>any</code> protocol expands to TCP and UDP	Two expressions	Two	Pass
R-19	Logging can be disabled	No <code>log prefix</code>	Absent	Pass
R-20	Sort key places allows before drops	Allow first	Allow first	Pass

ID	Test	Expected result	Actual result	Status
R-21	Store round-trips rules across reload	Values preserved	Preserved	Pass
R-22	Store file is private (mode 0600)	Group/other bits clear	Clear	Pass
R-23	Duplicate rule reported; <code>allow_duplicate</code> escape hatch works	Conflict then success	Correct	Pass
R-24	Contradiction (allow then shadowed drop) detected	Contradiction found	Found	Pass
R-25	Remove and toggle persist	State persists	Persists	Pass
R-26	Corrupt state file reported with filename	<code>SentinelFWError</code> naming file	Raised	Pass
R-27	Full script declares table, chain, <code>policy</code> <code>accept</code>	All present	Present	Pass
R-28	Full script uses <code>flush chain</code> , never <code>flush table</code>	<code>flush chain</code> only	Correct	Pass
R-29	Script never touches other tables	Only <code>inet sentinelfw</code>	Correct	Pass
R-30	Disabled rules omitted from script	Omitted	Omitted	Pass
R-31	Script is balanced and free of stray quotes/ separators	Valid shape	Valid	Pass

18.2 Configuration

Table 11 — Configuration test results.

ID	Test	Expected result	Actual result	Status
C-01	Safe defaults when no file exists	Expected defaults	Correct	Pass
C-02	<code>require=True</code> raises when absent	<code>ConfigNotFoundError</code>	Raised	Pass
C-03	Saved config is mode 0600	Owner-only	0600	Pass
C-04	Template is commented and private	Commented, 0600	Correct	Pass
C-05	Template refuses to clobber without <code>force</code>	Refuses; <code>force</code> overwrites	Correct	Pass
C-06	Explicit values honoured; refs built	<code>inet lab</code> , <code>inet lab input</code>	Correct	Pass
C-07	Relative paths resolve against root	Absolute under root	Correct	Pass
C-08	Unknown nested keys ignored, not fatal	Default kept	Correct	Pass
C-09	YAML <code>!!python/object</code> tag rejected, no execution	<code>ConfigValidationError</code> ; file absent	Correct	Pass
C-10	Malformed YAML reported clearly	<code>ConfigValidationError</code>	Raised	Pass
C-11	Non-mapping document rejected	<code>ConfigValidationError</code>	Raised	Pass
C-12	Invalid values rejected (10 cases)	<code>ConfigValidationError</code>	Raised	Pass
C-13	Watch-port range validated	<code>ConfigValidationError</code>	Raised	Pass

ID	Test	Expected result	Actual result	Status
C-14	Permission warning detects loose mode; harden fixes	Warning names 644 ; then 0600	Correct	Pass
C-15	Env var selects config file	Chosen file used	Correct	Pass

18.3 Storage and detection

Table 12 — Storage and detection test results.

ID	Test	Expected result	Actual result	Status
M-01	Parse a DROP line	Fields populated; blocked=True	Correct	Pass
M-02	Parse an ACCEPT line as not blocked	blocked=False	Correct	Pass
M-03	Ignore unrelated kernel lines	None	None	Pass
M-04	Missing fields do not crash the parser	Partial event	Partial	Pass
M-05	TCP flags captured and condensed	S	S	Pass
M-06	Severity reflects watched ports	Watched high, other info	Correct	Pass
M-07	Parser statistics counted	Counts increment	Correct	Pass
M-08	Insert and read back an event	Row returned with id	Correct	Pass
M-09	Bulk insert and count (50 rows)	50	50	Pass
M-10	Stats split blocked and accepted	Correct counts	Correct	Pass
M-11	Stats ignore events outside window	0	0	Pass
M-12	Alerts de-duplicated by fingerprint	One row; count accumulates	Correct	Pass
M-13	Alert acknowledgement	Acked excluded by default	Correct	Pass
M-14	Alert severity filter	One high-or-above	Correct	Pass
M-15	Prune removes only old events	One removed	Correct	Pass
M-16	Rule hit counters accumulate	8	8	Pass
M-17	Export respects limit and window	Correct rows	Correct	Pass
M-18	Reset clears events and alerts	Empty	Empty	Pass
M-19	Schema version recorded	Present	Present	Pass
M-20	Port scan detected	port_scan alert	Raised	Pass
M-21	Quiet traffic raises nothing	No alerts	None	Pass
M-22	Brute force detected	brute_force alert	Raised	Pass
M-23	Repeated blocks detected	repeated_block alert	Raised	Pass
M-24	Watched-port probe detected	sensitive_port alert	Raised	Pass
M-25	Connection spike detected	connection_spike alert	Raised	Pass
M-26	Detection dry-run persists nothing	Alerts returned; none stored	Correct	Pass
M-27	Alert descriptions are actionable	Recommendations and rationale	Present	Pass
M-28	Build source from file	Source describes path	Correct	Pass

ID	Test	Expected result	Actual result	Status
M-29	Missing file source reported	SentinelFWError	Raised	Pass
M-30	Demo source produces parsable lines	>10 events; some blocked	Correct	Pass
M-31	Collector ingests and detects	Events stored; ≥ 1 cycle	Correct	Pass
M-32	Collector can skip detection	0 cycles	0	Pass
M-33	Collector stops at max events (15)	15	15	Pass

18.4 End-to-end CLI

Table 13 — End-to-end CLI test results.

ID	Test	Expected result	Actual result	Status
E-01	Global flags accepted either side of command (6 cases)	Flags parsed	Correct	Pass
E-02	<code>-y</code> and <code>--yes</code> equivalent	<code>yes=True</code>	Correct	Pass
E-03	<code>-v</code> maps to verbose	<code>verbose=True</code>	Correct	Pass
E-04	No command prints help	Exit 0	0	Pass
E-05	Group without action prints help	Exit 0	0	Pass
E-06	<code>doctor</code> runs without root	Exit 0 or 6	6	Pass
E-07	<code>config init</code> creates a private file	0600 file	0600	Pass
E-08	<code>config init</code> refuses to clobber; <code>--force</code> works	Refuses then succeeds	Correct	Pass
E-09	<code>config path --json</code> reports in-project locations	All under project	Correct	Pass
E-10	<code>config check</code> accepts valid / rejects broken	0 / 4	Correct	Pass
E-11	Rules recorded but not applied	Output says not applied; file has IP	Correct	Pass
E-12	<code>--dry-run</code> writes nothing	No <code>rules.yaml</code>	Absent	Pass
E-13	<code>preview</code> never executes nft	"Nothing ... executed"	Correct	Pass
E-14	Invalid port is a clean error (no traceback)	Exit 4	4	Pass
E-15	Invalid IP is a clean error	Exit 4	4	Pass
E-16	Rule lifecycle (disable/enable/remove/missing)	Correct codes	Correct	Pass
E-17	<code>firewall status --json</code> machine-readable	<code>stored_rules=1</code> , table correct	Correct	Pass
E-18	<code>enable-logging --dry-run</code> warns about noise	Warning present	Present	Pass
E-19	Demo run populates the database	Status JSON works	Correct	Pass
E-20	JSON output is a single valid document (5 cases)	Parsable JSON	Correct	Pass

ID	Test	Expected result	Actual result	Status
E-21	Demo data labelled synthetic	"synthetic" present	Present	Pass
E-22	Ingest arbitrary log file	"Imported 1 event"	Correct	Pass
E-23	<code>alerts list --json</code> serialisable	<code>count</code> , <code>alerts</code>	Correct	Pass
E-24	Alert ids discoverable	Status and list work	Correct	Pass
E-25	<code>monitor reset</code> clears events	Status still works	Correct	Pass
E-26	<code>detect --dry-run</code> persists nothing	Exit 0	0	Pass
E-27	Report generates in every format	Text, brief, JSON	Correct	Pass
E-28	Report save writes into project	Private <code>.md</code> created	Correct	Pass
E-29	Report list shows readable timestamp	No raw epoch	Correct	Pass
E-30	Dashboard snapshot JSON serialisable	Parsable	Correct	Pass
E-31	Dashboard renders once	"SentinelFW" present	Present	Pass
E-32	<code>explain port</code> (22/3389/443)	Service names	Correct	Pass
E-33	<code>explain port</code> rejects out-of-range cleanly	Exit 0 or 4	Correct	Pass
E-34	<code>explain attack</code> lists and details	<code>port_scan</code> ; "How to respond"	Correct	Pass
E-35	<code>explain event --last</code>	Exit 0	0	Pass
E-36	<code>explain event</code> without target rejected	Exit 4	4	Pass
E-37	<code>db stats</code> and <code>db export</code> (CSV/JSON)	Correct header/rows	Correct	Pass
E-38	<code>db export --output</code> private file	0600	0600	Pass
E-39	<code>monitor poll</code> terminates (does not follow)	< 20 s	Correct	Pass
E-40	<code>--yes</code> does not bypass duplicate check	Conflict then <code>--force</code>	Correct	Pass

18.5 Safety and containment

Table 14 — Safety and containment test results.

ID	Test	Expected result	Actual result	Status
S-01	<code>apply</code> without root is refused	Exit 3; "Root privileges"; nothing applied	Correct	Pass
S-02	<code>apply --dry-run</code> needs no root	Exit 0	0	Pass
S-03	Non-interactive mutation without <code>--yes</code> refused	Exit 5; no <code>rules.yaml</code>	Correct	Pass
S-04	Generated script contains no foreign table	Only <code>inet sentinelfw</code>	Correct	Pass
S-05	<code>flush chain</code> used, <code>flush table</code> never	<code>flush chain</code> only	Correct	Pass
S-06	Comment injection cannot add a statement	Balanced; no separator	Correct	Pass

ID	Test	Expected result	Actual result	Status
S-07	Custom rule injection rejected	<code>RuleValidationError</code>	Raised	Pass
S-08	YAML object deserialisation rejected	<code>ConfigValidationError</code> ; no side effect	Correct	Pass
S-09	Invalid IP/port never reach the script	Rejected before rendering	Correct	Pass
S-10	Corrupt rules file reported, not executed	<code>SentinelFWError</code>	Raised	Pass

18.6 Summary

Table 15 — Test summary by module.

Test file	Concern	Tests	Passed	Failed	Skipped
<code>test_cli.py</code>	CLI, exit codes, safety	57	57	0	0
<code>test_rules.py</code>	Validation, rendering, containment	46	46	0	0
<code>test_monitoring.py</code>	Parsing, storage, detection	33	33	0	0
<code>test_config.py</code>	Configuration and permissions	24	24	0	0
Total		160	160	0	0

The complete inventory of all 160 test cases is listed in Appendix D.

19. Performance Analysis

SentinelFW's hot paths are deliberately cheap. Rule management touches a small YAML file; detection runs as indexed SQL over a bounded window; parsing is a sequence of regular expressions. Measurements below were taken on the test environment in Table 9, using the median of repeated runs to reduce noise.

Two kinds of number matter. **In-process latency** (Table 16) isolates the work SentinelFW actually does. **End-to-end CLI latency** (Table 17) includes Python interpreter startup and process teardown, which dominate every command; this is the number a user feels.

19.1 In-process latency

Table 16 — In-process latency measurements (median / minimum over repeated runs).

Operation	Median	Minimum	Notes
Database insert (one event)	0.06 ms	0.05 ms	WAL, autocommit-friendly
nft script render (full ruleset)	0.02 ms	0.02 ms	Pure string building
Rule add + remove (atomic)	2.1 ms	1.5 ms	Temp-file-and-rename + rewrite
Parse 1000 log lines	160 ms	148 ms	≈ 6,300 lines/s
Bulk insert 1000 events	10.2 ms	8.5 ms	Single transaction

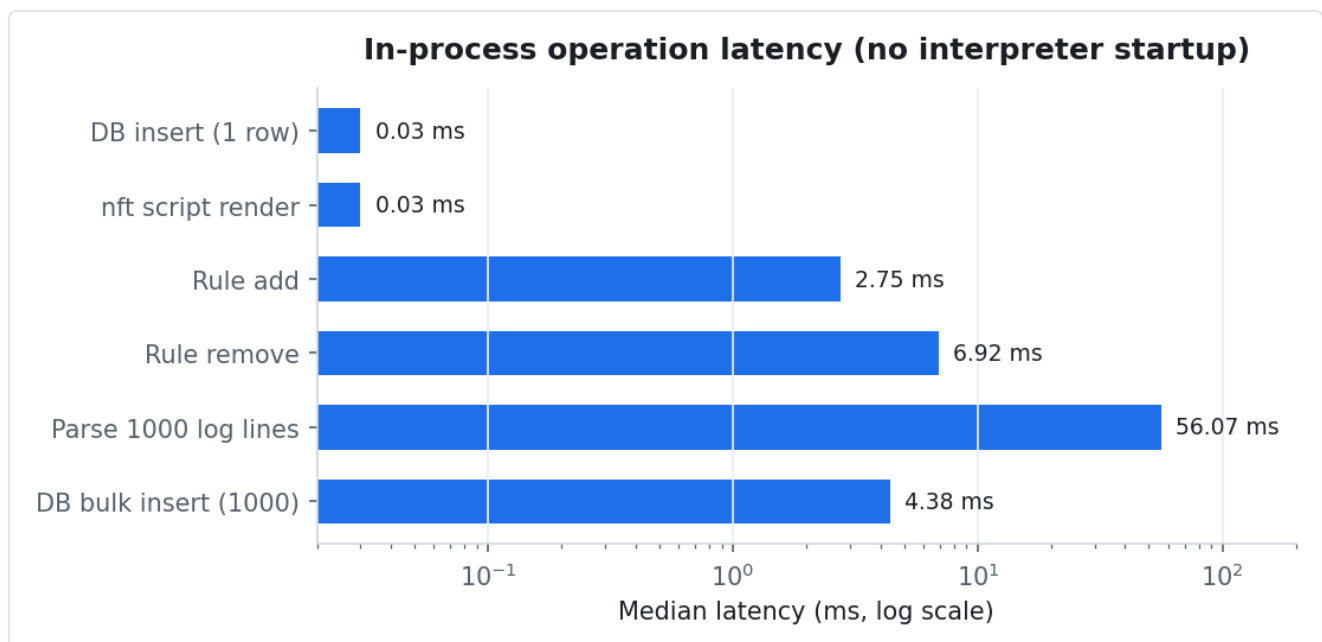


Figure 11 — In-process operation latency (log scale).

19.2 End-to-end CLI latency

Table 17 — End-to-end CLI command latency (median wall-clock, includes interpreter start).

Command	Median	Minimum
<code>firewall block-ip</code>	320 ms	308 ms
<code>firewall remove ID</code>	311 ms	306 ms

Command	Median	Minimum
<code>firewall preview</code>	314 ms	305 ms
<code>report generate</code>	327 ms	316 ms
<code>dashboard --once</code>	332 ms	319 ms
<code>monitor status</code>	320 ms	310 ms
<code>alerts list</code>	320 ms	309 ms

The spread between commands is small; roughly 300 ms of every figure is the fixed cost of starting a Python interpreter and importing Rich and PyYAML. The tool's own work is a few milliseconds. A single detection cycle over the 67-event demo window completed in **0.002 s**, and a full 8-second demo ingestion of 67 events used a single batch write.

19.3 Scaling characteristics

Detection cost scales with the **window**, not the size of the history, because every detector is an indexed aggregate query bounded by time. Storage cost is kept flat by batched writes and by retention pruning. The dashboard renders from a bounded set of queries (top sources, top ports, recent alerts, latest events) rather than from the full event table.

20. Security Analysis

SentinelFW is a security tool, so its own security posture matters. This section states the threat model and the controls, each of which maps to one or more tests.

20.1 Threat model

- **Malicious input** — an operator is tricked into blocking a crafted "address" or writing a crafted comment that escapes the generated nft script.
- **Configuration as a weapon** — a hostile `config.yaml` or `rules.yaml` is read and executes code.
- **Confirmation bypass** — a mutation proceeds without a human decision because stdin is not a terminal.
- **Containment failure** — a bug causes SentinelFW to modify or flush a table it does not own, breaking Docker, UFW or the host's connectivity.
- **Library misuse** — a shell is invoked with a string built from user input.

20.2 Controls

Table 19 — Security controls and their tests.

Control	Implementation	Tests
Input validation	<code>ipaddress</code> for addresses/networks; integer range check for ports; fixed protocol set	R-01...R-11
Injection prevention	Comment sanitiser (strips quotes, backslashes, newlines, <code>;</code>); custom rules reject newline/ <code>;</code>	R-12...R-16, S-06, S-07
Containment	Only <code>table inet sentinelfw</code> is ever written; <code>nft_manager</code> refuses any other table	R-28, R-29, S-04
Atomic application	Single <code>nft -f -</code> transaction; <code>flush chain</code> , never <code>flush table</code> ; verify by read-back	R-28, S-05
Safe deserialisation	<code>yaml.safe_load</code> only, everywhere; no <code>yaml.load / FullLoader</code>	C-09, S-08
No shell	<code>subprocess.run(argv)</code> with argument lists; no <code>shell=True</code>	Reviewed; S-09
Least privilege	Root only for <code>apply</code> , <code>validate</code> , <code>backup</code> , <code>restore</code> , journal reads	S-01, S-02
Explicit consent	Non-interactive mutation refused unless <code>--yes</code>	S-03
Confirmation is not bypass of checks	<code>--yes</code> suppresses prompts only; <code>--force</code> is separate	E-40
File permissions	<code>config.yaml</code> and <code>rules.yaml</code> at 0600; exports private; <code>config harden</code>	C-03, C-14, E-07, E-38
Audit logging	Every <code>nft</code> invocation logged to <code>logs/sentinelfw.log</code> by a rotating handler	Logging review
Honest destructive operations	<code>restore</code> states it flushes the entire ruleset; no quieter flag	Docs + CLI text

Control	Implementation	Tests
Error handling	One exception class per failure, each with a hint; no tracebacks at the CLI	E-14, E-15, R-26, C-10

20.3 What SentinelFW does not protect

It is worth being explicit. SentinelFW filters in *one* chain, at priority `-10`, with an `accept` policy; it is an additional layer, not a hardened perimeter. It sees only what the kernel logged, so it does not capture packets or inspect payloads. Findings are heuristics, not proof of compromise. The event database and reports contain source addresses, ports and timestamps and should be treated as sensitive; they are gitignored, and `config harden` tightens the config and rules files.

21. Code Coverage

Coverage was measured with `pytest-cov` across the application package. Overall statement coverage is **72%**. Coverage is highest where correctness is most safety-critical (detection 94%, configuration 85%, rules 82%, database 82%) and lowest in the root-only and error branches that the unprivileged suite deliberately cannot execute (`firewall/backup.py` 30%, `firewall/nft_manager.py` 45%). That distribution is expected and intentional: the untested lines are precisely the ones that would touch a live firewall.

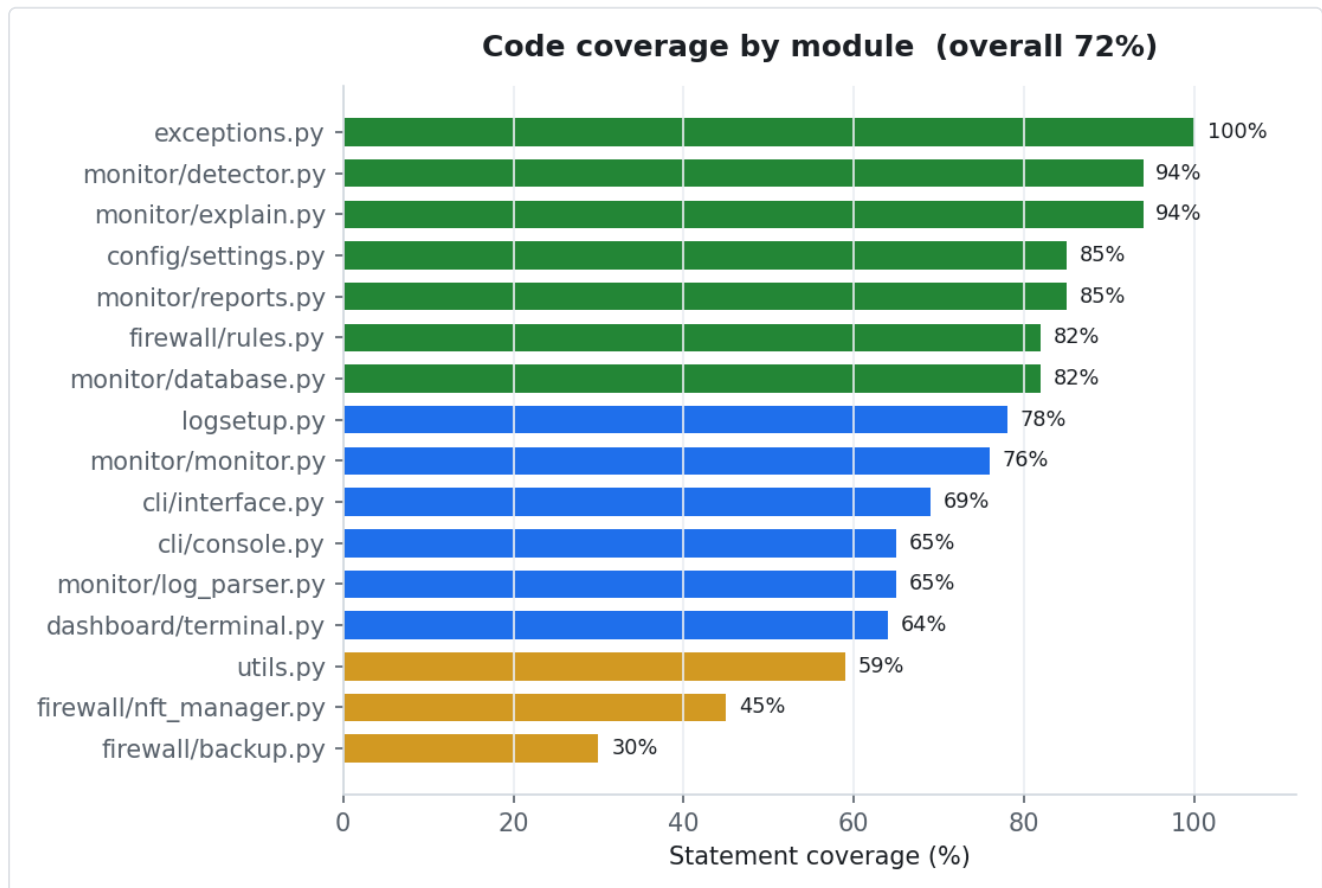


Figure 9 — Code coverage by module (overall 72%).

Table 18 — Coverage by module.

Module	Statements	Missed	Coverage
<code>exceptions.py</code>	54	0	100%
<code>monitor/detector.py</code>	223	14	94%
<code>monitor/explain.py</code>	110	7	94%
<code>config/settings.py</code>	330	48	85%
<code>monitor/reports.py</code>	294	44	85%
<code>firewall/rules.py</code>	373	68	82%
<code>monitor/database.py</code>	365	64	82%
<code>logsetup.py</code>	51	11	78%
<code>monitor/monitor.py</code>	183	44	76%

Module	Statements	Missed	Coverage
cli/interface.py	1110	347	69%
cli/console.py	129	45	65%
monitor/log_parser.py	373	130	65%
dashboard/terminal.py	210	76	64%
utils.py	215	89	59%
firewall/nft_manager.py	279	154	45%
firewall/backup.py	141	99	30%
Total	4460	1240	72%

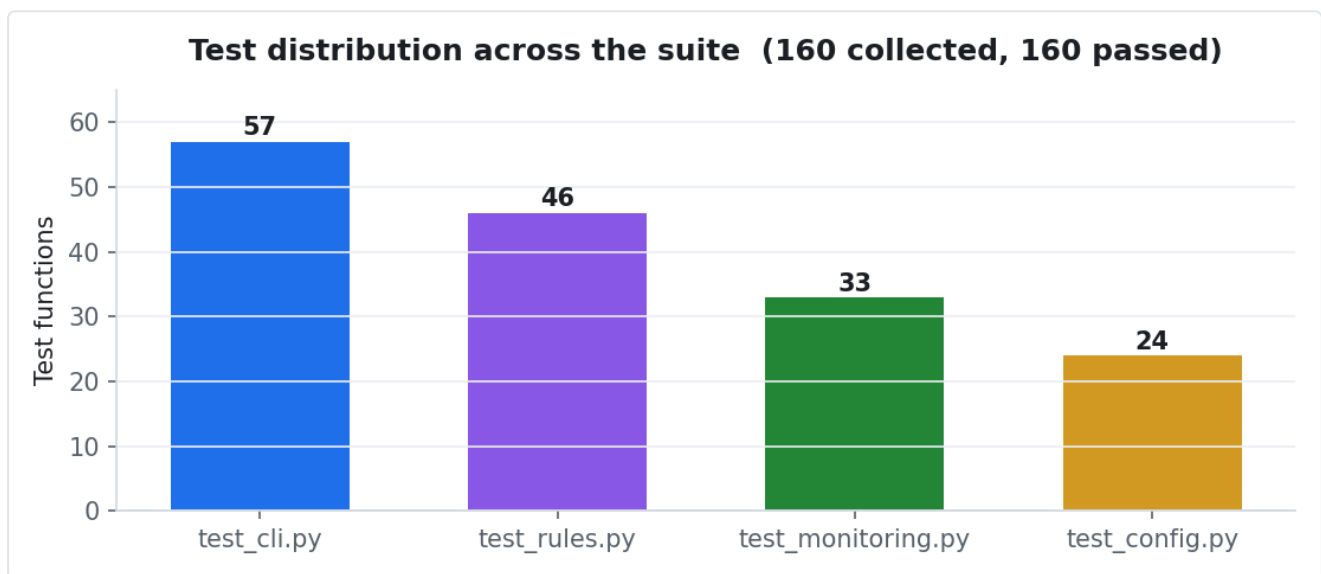


Figure 10 — Test distribution across the suite.

22. Limitations

Table 20 — Known limitations.

#	Limitation	Consequence
L-1	Not a complete firewall	It manages one chain; a full policy still needs nftables or a distro tool
L-2	Linux and nftables only	No Windows, macOS, pf or iptables backend
L-3	Log-based detection	Anything the kernel does not log cannot be detected
L-4	Heuristic detection	Severity comes from volume and patterns, not payload inspection; false positives are possible
L-5	Thresholds need tuning	Defaults suit a single-purpose host, not a busy server
L-6	Restore is destructive	<code>firewall restore</code> flushes the entire ruleset, including Docker's and UFW's
L-7	Demo data is fabricated	<code>monitor demo</code> writes to the real database; must be reset afterwards
L-8	Root-only branches untested at runtime	The suite proves refusal and generation, not live <code>nft</code> acceptance
L-9	Single host	No remote or multi-host monitoring
L-10	No auto-remediation	It advises; it does not silently ban or change rules

The last point is a deliberate design choice rather than a shortcoming. Silent auto-banning is how a tool blocks an address that turns out to matter; SentinelFW keeps a human in the loop.

23. Future Enhancements

Table 21 — Future enhancements.

#	Enhancement	Rationale
F-1	Machine-learning anomaly detection	Complement thresholds with a baseline of normal traffic per host
F-2	Email / Telegram / webhook alerts	Push findings to the operator instead of requiring a poll
F-3	Multi-host collection	A central collector aggregating several SentinelFW agents
F-4	Threat-intelligence feeds	Enrich source IPs with reputation data
F-5	Web dashboard	Browser access for teams
F-6	Container monitoring	Attribute events to Docker/Podman workloads
F-7	Automatic rule suggestions	Generate a block rule from a high-severity alert with one command
F-8	Signed rule bundles / export/import	Share vetted rulesets between machines
F-9	Optional eBPF collector	Reduce dependence on kernel log volume
F-10	Pluggable detectors	A documented detector interface for community rules

24. Conclusion

SentinelFW set out to make two tasks that are usually separate — managing a firewall and understanding what it does — into one coherent, safe, teachable tool. It achieved that by refusing to take shortcuts at the points that matter.

Rules never reach the kernel by accident: changes are recorded in a readable file, previewed in full, backed up, confirmed, applied as a single atomic transaction, and verified by reading the kernel back. The tool owns exactly one object and refuses to touch any other, so a bug cannot destroy an unrelated ruleset. Input is validated and sanitised before it reaches the generated script, configuration is parsed as data rather than code, and nothing is ever handed to a shell. Consent is required, and `--yes` deliberately does not bypass the safety checks.

On the monitoring side, the tool turns opaque kernel log lines into structured events, applies five transparent and tunable detectors, de-duplicates findings, and explains each one in plain language with concrete next steps. A dashboard, three report formats and a knowledge base make the result usable.

The project is backed by **160 passing tests** in about **51 seconds**, **72% statement coverage** concentrated on the correctness-critical modules, and a test suite that is itself safe: it never invokes `nft` and never needs root. Measured performance is dominated by Python interpreter startup, not by SentinelFW's own work, which completes in milliseconds.

What remains is deliberate. SentinelFW is a manager, not a firewall; it is single-host by design; its detection is heuristic; and it advises rather than auto-remediates. Those are boundaries, honestly stated, not flaws hidden. Within them, SentinelFW does what it promised: it makes a firewall change inspectable, reversible and explained.

25. References

1. The netfilter project. *nftables wiki*. <https://wiki.nftables.org/>
2. The netfilter project. *nft(8) manual page*.
3. Python Software Foundation. *Python 3 documentation*. <https://docs.python.org/3/>
4. Python Software Foundation. *ipaddress — IPv4/IPv6 manipulation library*.
5. SQLite Consortium. *SQLite documentation*. <https://sqlite.org/docs.html>
6. PyYAML. *PyYAML documentation*. <https://pyyaml.org/wiki/PyYAMLDocumentation>
7. Textualize. *Rich documentation*. <https://rich.readthedocs.io/>
8. pytest development team. *pytest documentation*. <https://docs.pytest.org/>
9. systemd project. *journalctl(1) manual page*.
10. Keep a Changelog. <https://keepachangelog.com/>
11. Semantic Versioning. <https://semver.org/>
12. OWASP Foundation. *Command Injection and Deserialization Cheat Sheets*.
13. SentinelFW. *Safety model (docs/SAFETY.md)*. Repository documentation.

Appendix A — Installation

Everything on Kali, Debian and Ubuntu comes from `apt`:

```
sudo apt update
sudo apt install -y nftables python3-rich python3-yaml
```

Run it straight from the clone, with nothing to install:

```
git clone https://github.com/Lucky-Joshi/SentinelFW.git
cd SentinelFW
./sentinelfw doctor
```

Optional system-wide install (Kali's Python is externally managed, so use `pipx`):

```
sudo apt install -y pipx
pipx install .
sentinelfw doctor
```

Appendix B — First Session

```
./sentinelfw doctor                # check the machine
./sentinelfw config init           # create config.yaml (mode 0600)
./sentinelfw monitor demo --seconds 30 # synthetic traffic, no root
./sentinelfw dashboard --once
./sentinelfw alerts list --explain
./sentinelfw report generate

./sentinelfw firewall block-ip 203.0.113.7 -r "scanner" # records only
./sentinelfw firewall preview                          # exact nft script
sudo ./sentinelfw firewall apply                        # atomic, verified
```

Appendix C — Sample Configuration

```
firewall:
  table: sentinelfw
  family: inet
  chain: input
  hook: input
  priority: -10
  policy: accept
  log_enabled: true
  log_level: info
  log_prefix: SENTINELFW

database:
  path: database/events.db
  retention_days: 90

monitoring:
  source: journal          # journal | file | demo
  log_files:
    - /var/log/kern.log
    - /var/log/syslog

detection:
  port_scan_threshold: 20
  brute_force_threshold: 100
  spike_threshold: 300
  repeated_block_threshold: 50
  alert_cooldown_seconds: 900
```

Appendix D — Full Test Inventory

The following table lists all 160 tests. Every test passed.

#	File	Test node id	Result
1	tests/test_cli.py	test_global_flags_work_on_either_side_of_the_command[argv0]	PASS
2	tests/test_cli.py	test_global_flags_work_on_either_side_of_the_command[argv1]	PASS
3	tests/test_cli.py	test_global_flags_work_on_either_side_of_the_command[argv2]	PASS
4	tests/test_cli.py	test_global_flags_work_on_either_side_of_the_command[argv3]	PASS
5	tests/test_cli.py	test_global_flags_work_on_either_side_of_the_command[argv4]	PASS
6	tests/test_cli.py	test_global_flags_work_on_either_side_of_the_command[argv5]	PASS
7	tests/test_cli.py	test_short_and_long_yes_are_equivalent	PASS
8	tests/test_cli.py	test_verbose_short_flag_maps_to_verbose	PASS
9	tests/test_cli.py	test_no_command_prints_help	PASS
10	tests/test_cli.py	test_group_without_action_prints_subcommand_help	PASS
11	tests/test_cli.py	test_doctor_runs_without_root	PASS
12	tests/test_cli.py	test_config_init_creates_a_private_file	PASS
13	tests/test_cli.py	test_config_init_refuses_to_clobber	PASS
14	tests/test_cli.py	test_config_path_reports_locations	PASS
15	tests/test_cli.py	test_config_check_accepts_a_valid_file	PASS
16	tests/test_cli.py	test_config_check_rejects_a_broken_file	PASS
17	tests/test_cli.py	test_rules_are_recorded_but_not_applied	PASS
18	tests/test_cli.py	test_dry_run_writes_nothing	PASS
19	tests/test_cli.py	test_preview_never_executes_nft	PASS
20	tests/test_cli.py	test_invalid_port_is_a_clean_error	PASS
21	tests/test_cli.py	test_invalid_ip_is_a_clean_error	PASS
22	tests/test_cli.py	test_apply_without_root_is_refused	PASS
23	tests/test_cli.py	test_apply_dry_run_needs_no_root	PASS
24	tests/test_cli.py	test_non_interactive_mutation_without_yes_is_refused	PASS
25	tests/test_cli.py	test_rule_lifecycle	PASS
26	tests/test_cli.py	test_firewall_status_json_is_machine_readable	PASS
27	tests/test_cli.py	test_syn_logging_dry_run_warns_about_noise	PASS
28	tests/test_cli.py	test_demo_run_populates_the_database	PASS
29	tests/test_cli.py	test_json_output_is_a_single_valid_document[argv0-total_events]	PASS
30	tests/test_cli.py	test_json_output_is_a_single_valid_document[argv1-stored_rules]	PASS
31	tests/test_cli.py	test_json_output_is_a_single_valid_document[argv2-count]	PASS
32	tests/test_cli.py	test_json_output_is_a_single_valid_document[argv3-total_events]	PASS
33	tests/test_cli.py	test_json_output_is_a_single_valid_document[argv4-range]	PASS
34	tests/test_cli.py	test_demo_data_is_labelled_as_synthetic	PASS
35	tests/test_cli.py	test_ingest_reads_an_arbitrary_log_file	PASS

#	File	Test node id	Result
36	tests/test_cli.py	test_alerts_list_is_json_serialisable	PASS
37	tests/test_cli.py	test_alert_ids_are_discoverable	PASS
38	tests/test_cli.py	test_monitor_reset_clears_events	PASS
39	tests/test_cli.py	test_detect_dry_run_persists_nothing	PASS
40	tests/test_cli.py	test_report_generates_in_every_format	PASS
41	tests/test_cli.py	test_report_save_writes_into_the_project	PASS
42	tests/test_cli.py	test_report_list_shows_a_readable_timestamp	PASS
43	tests/test_cli.py	test_dashboard_snapshot_is_json_serialisable	PASS
44	tests/test_cli.py	test_dashboard_renders_once	PASS
45	tests/test_cli.py	test_explain_port[22-ssh]	PASS
46	tests/test_cli.py	test_explain_port[3389-rdp]	PASS
47	tests/test_cli.py	test_explain_port[443-https]	PASS
48	tests/test_cli.py	test_explain_port_rejects_out_of_range[0]	PASS
49	tests/test_cli.py	test_explain_port_rejects_out_of_range[65536]	PASS
50	tests/test_cli.py	test_explain_port_rejects_out_of_range[70000]	PASS
51	tests/test_cli.py	test_explain_attack_lists_and_details	PASS
52	tests/test_cli.py	test_explain_event_last	PASS
53	tests/test_cli.py	test_explain_event_without_target_is_rejected	PASS
54	tests/test_cli.py	test_db_stats_and_export	PASS
55	tests/test_cli.py	test_db_export_to_file_is_private	PASS
56	tests/test_cli.py	test_monitor_poll_terminates_instead_of_following	PASS
57	tests/test_cli.py	test_yes_does_not_bypass_the_duplicate_check	PASS
58	tests/test_config.py	test_defaults_when_no_file	PASS
59	tests/test_config.py	test_require_raises_when_absent	PASS
60	tests/test_config.py	test_saved_config_is_owner_only	PASS
61	tests/test_config.py	test_template_is_commented_and_private	PASS
62	tests/test_config.py	test_template_refuses_to_clobber_without_force	PASS
63	tests/test_config.py	test_explicit_values_are_honoured	PASS
64	tests/test_config.py	test_relative_paths_resolve_against_root	PASS
65	tests/test_config.py	test_nested_unknown_keys_are_ignored_not_fatal	PASS
66	tests/test_config.py	test_yaml_is_parsed_safely	PASS
67	tests/test_config.py	test_malformed_yaml_is_reported_clearly	PASS
68	tests/test_config.py	test_non_mapping_document_is_rejected	PASS
69	tests/test_config.py	test_invalid_values_are_rejected[firewall:\n table: 'bad name'\n]	PASS
70	tests/test_config.py	test_invalid_values_are_rejected[firewall:\n family: ethernet\n]	PASS
71	tests/test_config.py	test_invalid_values_are_rejected[firewall:\n policy: reject\n]	PASS
72	tests/test_config.py	test_invalid_values_are_rejected[firewall:\n priority: 9999\n]	PASS
73	tests/test_config.py	test_invalid_values_are_rejected[firewall:\n log_level: verbose\n]	PASS

#	File	Test node id	Result
74	tests/test_config.py	test_invalid_values_are_rejected[firewall:\n log_prefix: 'has space'\n]	PASS
75	tests/test_config.py	test_invalid_values_are_rejected[monitoring:\n source: carrier-pigeon\n]	PASS
76	tests/test_config.py	test_invalid_values_are_rejected[detection:\n port_scan_threshold: 1\n]	PASS
77	tests/test_config.py	test_invalid_values_are_rejected[detection:\n port_scan_window_seconds: 1\n]	PASS
78	tests/test_config.py	test_invalid_values_are_rejected[database:\n retention_days: -1\n]	PASS
79	tests/test_config.py	test_watch_ports_range_is_validated	PASS
80	tests/test_config.py	test_permissions_warning_detects_loose_mode	PASS
81	tests/test_config.py	test_env_var_selects_config_file	PASS
82	tests/ test_monitoring.py	test_parsing_a_drop_line	PASS
83	tests/ test_monitoring.py	test_parsing_an_accept_line_as_not_blocked	PASS
84	tests/ test_monitoring.py	test_ignores_unrelated_kernel_lines	PASS
85	tests/ test_monitoring.py	test_missing_fields_do_not_crash_the_parser	PASS
86	tests/ test_monitoring.py	test_tcp_flags_are_captured	PASS
87	tests/ test_monitoring.py	test_severity_reflects_watched_ports	PASS
88	tests/ test_monitoring.py	test_parser_statistics_are_counted	PASS
89	tests/ test_monitoring.py	test_insert_and_read_back	PASS
90	tests/ test_monitoring.py	test_bulk_insert_and_count	PASS
91	tests/ test_monitoring.py	test_stats_splits_blocked_and_accepted	PASS
92	tests/ test_monitoring.py	test_stats_ignores_events_outside_the_window	PASS
93	tests/ test_monitoring.py	test_alerts_are_deduplicated_by_fingerprint	PASS
94	tests/ test_monitoring.py	test_alert_acknowledgement	PASS
95	tests/ test_monitoring.py	test_alert_severity_filter	PASS
96	tests/ test_monitoring.py	test_prune_removes_only_old_events	PASS
97	tests/ test_monitoring.py	test_rule_hit_counters	PASS
98	tests/ test_monitoring.py	test_export_rows_respects_limit_and_window	PASS

#	File	Test node id	Result
99	tests/ test_monitoring.py	test_reset_clears_everything	PASS
100	tests/ test_monitoring.py	test_schema_version_is_recorded	PASS
101	tests/ test_monitoring.py	test_port_scan_is_detected	PASS
102	tests/ test_monitoring.py	test_quiet_traffic_raises_nothing	PASS
103	tests/ test_monitoring.py	test_brute_force_on_one_port_is_detected	PASS
104	tests/ test_monitoring.py	test_repeated_blocks_are_detected	PASS
105	tests/ test_monitoring.py	test_watched_port_probe_is_detected	PASS
106	tests/ test_monitoring.py	test_connection_spike_is_detected	PASS
107	tests/ test_monitoring.py	test_detection_dry_run_persists_nothing	PASS
108	tests/ test_monitoring.py	test_alert_descriptions_are_actionable	PASS
109	tests/ test_monitoring.py	test_build_source_from_file	PASS
110	tests/ test_monitoring.py	test_missing_file_source_is_reported	PASS
111	tests/ test_monitoring.py	test_demo_source_produces_parsable_lines	PASS
112	tests/ test_monitoring.py	test_collector_ingests_and_detects	PASS
113	tests/ test_monitoring.py	test_collector_can_skip_detection	PASS
114	tests/ test_monitoring.py	test_collector_run_forever_stops_at_max_events	PASS
115	tests/test_rules.py	test_valid_ip_or_network[1.2.3.4-1.2.3.4]	PASS
116	tests/test_rules.py	test_valid_ip_or_network[10.0.0.0/16-10.0.0.0/16]	PASS
117	tests/test_rules.py	test_valid_ip_or_network[2001::db8::1-2001::db8::1]	PASS
118	tests/test_rules.py	test_valid_ip_or_network[8.8.8.8 -8.8.8.8]	PASS
119	tests/test_rules.py	test_invalid_ip_values_are_rejected[]	PASS
120	tests/test_rules.py	test_invalid_ip_values_are_rejected[]	PASS
121	tests/test_rules.py	test_invalid_ip_values_are_rejected[not-an-ip]	PASS
122	tests/test_rules.py	test_invalid_ip_values_are_rejected[999.1.1.1]	PASS
123	tests/test_rules.py	test_invalid_ip_values_are_rejected[1.2.3]	PASS
124	tests/test_rules.py	test_invalid_ip_values_are_rejected[1.2.3.4/33]	PASS
125	tests/test_rules.py	test_invalid_ip_values_are_rejected[None]	PASS
126	tests/test_rules.py	test_absurdly_broad_networks_are_rejected[0.0.0.0/0]	PASS
127	tests/test_rules.py	test_absurdly_broad_networks_are_rejected[10.0.0.0/4]	PASS

#	File	Test node id	Result
128	tests/test_rules.py	test_valid_ports[22-22]	PASS
129	tests/test_rules.py	test_valid_ports[443-443]	PASS
130	tests/test_rules.py	test_valid_ports[8080 -8080]	PASS
131	tests/test_rules.py	test_invalid_ports_are_rejected[0]	PASS
132	tests/test_rules.py	test_invalid_ports_are_rejected[65536]	PASS
133	tests/test_rules.py	test_invalid_ports_are_rejected[-1]	PASS
134	tests/test_rules.py	test_invalid_ports_are_rejected[abc]	PASS
135	tests/test_rules.py	test_invalid_ports_are_rejected[22.5]	PASS
136	tests/test_rules.py	test_invalid_ports_are_rejected[]	PASS
137	tests/test_rules.py	test_invalid_ports_are_rejected[None]	PASS
138	tests/test_rules.py	test_protocol_normalisation	PASS
139	tests/test_rules.py	test_port_service_name_is_best_effort	PASS
140	tests/test_rules.py	test_ip_rule_target	PASS
141	tests/test_rules.py	test_port_rule_target_names_the_service	PASS
142	tests/test_rules.py	test_unknown_kind_or_action_is_rejected	PASS
143	tests/test_rules.py	test_custom_rule_rejects_statement_smuggling	PASS
144	tests/test_rules.py	test_comment_cannot_break_out_of_the_nft_quotes	PASS
145	tests/test_rules.py	test_comment_newlines_cannot_smuggle_a_second_rule	PASS
146	tests/test_rules.py	test_sanitized_comment_is_used	PASS
147	tests/test_rules.py	test_rendered_rule_contains_expected_tokens	PASS
148	tests/test_rules.py	test_protocol_any_expands_to_tcp_and_udp	PASS
149	tests/test_rules.py	test_logging_can_be_disabled	PASS
150	tests/test_rules.py	test_sort_key_puts_allows_before_drops	PASS
151	tests/test_rules.py	test_store_round_trips_rules	PASS
152	tests/test_rules.py	test_store_file_is_private	PASS
153	tests/test_rules.py	test_duplicate_rule_is_reported	PASS
154	tests/test_rules.py	test_contradiction_is_detected	PASS
155	tests/test_rules.py	test_remove_and_toggle	PASS
156	tests/test_rules.py	test_corrupt_state_file_is_reported	PASS
157	tests/test_rules.py	test_full_script_declares_table_and_chain	PASS
158	tests/test_rules.py	test_script_never_touches_other_tables	PASS
159	tests/test_rules.py	test_disabled_rules_are_omitted	PASS
160	tests/test_rules.py	test_script_is_valid_nft_syntax_shaped	PASS

Appendix E — Sample Logs and Evidence

Example firewall log lines the parser recognises (the prefix is what it looks for):

```
Oct  4 03:10:02 kali kernel: [SENTINELFW-DROP ] IN=eth0 OUT= MAC= SRC=203.0.113.9
DST=198.51.100.1 LEN=40 PROTO=TCP SPT=40000 DPT=22
Oct  4 03:10:03 kali kernel: [SENTINELFW-ACCEPT ] IN=eth0 OUT= MAC= SRC=198.51.100.7
DST=198.51.100.1 LEN=60 PROTO=TCP SPT=51000 DPT=443
```

The generated nft script for the rules recorded during the verification session (three rules), as produced by `firewall preview`:

```
add table inet sentinelfw
add chain inet sentinelfw input { type filter hook input priority -10; policy accept; }
flush chain inet sentinelfw input
add rule inet sentinelfw input ip saddr 198.51.100.0/24 counter log prefix "SENTINELFW-ACCEPT "
level info accept comment "lab net"
add rule inet sentinelfw input ip saddr 203.0.113.7 counter log prefix "SENTINELFW-DROP " level
info drop comment "scanner"
add rule inet sentinelfw input tcp dport 23 counter log prefix "SENTINELFW-DROP " level info
drop comment "telnet off"
```

Observed monitor statistics after an 8-second demo run:

```
{
  "total_events": 67,
  "blocked_events": 67,
  "accepted_events": 0,
  "unique_sources": 6,
  "unique_ports": 16,
  "alert_total": 3,
  "alerts_by_severity": { "high": 3 }
}
```

End of report.